Expert Systems With Applications 207 (2022) 117936

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Generating realistic cyber data for training and evaluating machine learning classifiers for network intrusion detection systems

Marc Chalé *, Nathaniel D. Bastian

Department of Operational Sciences, Air Force Institute of Technology, 2950 Hobson Way, Wright-Patterson AFB, 45433, OH, United States
Army Cyber Institute, United States Military Academy, 2101 New South Post Rd, West Point, 10996, NY, United States

ARTICLE INFO

Keywords:
Generative machine learning
Generative adversarial network
Variational autoencoder
Synthetic data generation
Network intrusion detection

ABSTRACT

Cyberspace operations, in conjunction with artificial intelligence and machine learning enhanced cyberspace infrastructure, make it possible to connect sensors directly to shooters independent of human control. These technologies serve as the pivot around which cyber data from the military's Internet of Battlefield Things, for example, will be turned into actionable insight and knowledge and, ultimately, an information advantage for the military. As such, network intrusion detection systems must detect, evaluate, and respond to malicious cyber traffic at machine speed. Generative adversarial networks and variational autoencoders are fit as generative models with labeled cyber data from a real military enterprise network. These generative models are used to create realistic, synthetic cyber data. A combination of real and synthetic cyber data sets are then used to train several machine learning models for network intrusion detection. Purely synthetic data is shown to be statistically similar to the real data. There is no statistically significant difference in the performance of classifiers trained with real data versus a combination of real and synthetic data; however, classifiers trained with only synthetic data underperformed. To avoid a decrease in intrusion detection performance, classifiers must be trained with at least 15% real data.

1. Introduction

Cyberspace has characteristics that differ from the air, land, maritime, and space domains. These characteristics affect how the military operates and defends cyberspace infrastructure, information, information systems, and data. Military forces have integrated and synchronized cyberspace capabilities along with the authorities to conduct effective cyberspace operations as part of an overall combined arms strategy in support of multi-domain operations and joint all-domain command control. Cyberspace operations provide the capability to process and manage operationally relevant actions, allowing simultaneous and linked maneuver, in, through, and across multiple domains and the information environment (IE), while engaging adversaries and populations directly across time, space, and scale. The IE is the aggregate of individuals, organizations, and systems that collect, process, disseminate, or act on information (Cicalese, 2013). To manage the complexity of the cyberspace domain, the military has divided it into separate layers, to include physical, logical, and cyber-persona (United States Department of the Army, 2010). Connections between the layers of cyberspace generate a portion of the IE that is divided into three dimensions — physical, informational, and cognitive; each dimension is associated with a specific layer of cyberspace for which the latest artificial intelligence (AI) and machine learning (ML) technology can be integrated.

Cyberspace operations, in conjunction with AI and ML enhanced cyberspace infrastructure, make it possible to connect sensors directly to shooters independent of human control. Current military networks can best be described as sensor-to-human-to-shooter with this construct used to keep a human decision-maker as the final arbiter regarding the use of lethal force. Cyberspace infrastructure is network agnostic as it supports all users. From intelligence to fire control to aviation to mission command, all networks of purpose defined by the information to be exchanged can simultaneously coexist within the framework of cyberspace infrastructure. In a future with more prevalent semi-independent and independent information-centric technologies that leverage AI and ML, competitive advantage is gained since network connections will be ad hoc and information exchange and interconnectivity will fluctuate at speeds beyond human abilities to manage and control.

Increased digitalization, a proliferation of new sensors, new communication modes, the internet-of-things and virtualization of social

* Corresponding author at: Department of Operational Sciences, Air Force Institute of Technology, 2950 Hobson Way, Wright-Patterson AFB, 45433, OH, United States.
E-mail addresses: marc.chale@afit.edu (M. Chalé), nathaniel.bastian@westpoint.edu (N.D. Bastian).

https://doi.org/10.1016/j.eswa.2022.117936
Received 3 January 2022; Received in revised form 28 May 2022; Accepted 19 June 2022
Available online 27 June 2022
0957-4174/Published by Elsevier Ltd.

communication have contributed significantly to formation of the military's Internet of Battlefield Things (IoBT). These AI and ML technologies serve as the pivot around which IoBT big data will be turned into actionable insight and knowledge and, ultimately, an information advantage for the military. As a component of the IoBT, intrusion detection systems (IDS), and specifically IDS embedded inside a network (NIDS), must detect, evaluate, and respond before a human operator may understand and react. These technologies enable the collection of synchronized, real-time capabilities to discover, define, analyze, and mitigate cyber threats and vulnerabilities without direct human intervention (Ventre, 2020). Quality data is needed to develop an ML-based NIDS that learns and manages network topologies, identifies and manages trusted users, detects network anomalies, identifies threats, and undertakes mitigation and response action. However, availability of and access to labeled cyber data is extremely limited, highlighting the need to leverage novel generative methods for deriving realistic cyber data for training and evaluating ML-based NIDS.

2. Literature review

In this section, a broader cybersecurity context is provided, to include issues and progress in cyber data. Related works in data augmentation, data sampling, and generative methods are discussed.

2.1. Cybersecurity

The military considers cyberspace to be a domain of operations which is susceptible to military targeting much like other domains of operations (Brent, 2019). Cyber technologies permeate through today's battlefield to facilitate competencies such as command and control, communication, and navigation (Ventre, 2020). These technologies are undergoing a transformation as the IoBT emerges as a paradigm of war fighting (Azmoodeh, Dehghantanha, & Choo, 2019). The IoBT is the connected system of cyber–physical sensors, computers, and actuators used to carry out warfare (Russell & Abdelzaher, 2018). This strategy is poised to shorten latency in intelligence collection, decision making, and effect delivery by adding autonomy throughout information pipelines. Smart devices increase human productivity and improve implementation of the observe–orient–decide–act loop (Russell & Abdelzaher, 2018); autonomous devices replace humans from *in the loop*, instead allowing humans to supervise from *on the loop* or observe from *out of the loop* (Scharre, 2018). The IoBT carries with it a new cyber exposure that is only starting to be understood. Countless connected devices with sensors and actuators are vulnerable targets when not properly secured and monitored (Miller, O'Halloran, Pollman, & Feeley, 2019).

Cybersecurity is a discipline associated with promoting "integrity, availability, and confidentiality" of information on digital systems (Russell & Abdelzaher, 2018). Data integrity ensures that data is modified only by authorized users and system integrity ensures that system functionality is unimpaired. Availability describes a system that is accessible to authorized users upon request. Systems exhibit confidentiality when they correctly deny illegal attempts to access private information (Stallings, Brown, Bauer, & Bhattacharjee, 2012). Early worms sought to exploit computer resources across networks and the first virus was an academic exercise that demonstrated intentional damage to infected computers. Later examples of malware were developed to criminally manipulate, steal, or destroy data and to degrade systems (Chen & Robert, 2004).

Early surveys of cyber threats (Anderson, 1972, 1980) were sponsored by United States military organizations and concluded that cyber systems were vulnerable to attack and required preemptive security functionality such as auditing behavior, and enforcing tiered privileges. The framework for automating intrusion detection is developed in Denning (1987), Denning and Neumann (1985). Axelsson (2000) and Hindy, Brosset, Bayne, Seeam, Tachtatzis, Atkinson, and Bellekens (2020) provide comprehensive reviews of intrusion detection technology. The United States Air Force deployed the first known NIDS in 1988. Today, NIDS are a primary component of the broader practices in network security monitoring. NIDS may be signature based, meaning they seek to identify patterns associated with known threats, or anomaly based meaning they detect abnormal behavior.

2.2. Issues and progress in cyber data

NIDS data is collected though combination of physical taps and specialized switches installed in critical network locations. Additionally, data collection can occur within host devices such as clients, servers, and workstations, however, these software based strategies are less reliable in the long term. Cyber data *collection applications* include Argus, Netsniff-ng, PRADS, Suricata, and Bro (recently re-branded as Zeek). The specific data being recorded may be statistical meta features of cyber packet headers or, in some cases, the entire packet headers and payloads. Zeek also belongs to the class of applications called *presentation tools* which presents live and recorded cyber data to operators and generates alerts. Best practice is for a human analyst to interpret all alerts and oversee a response (Bejtlich, 2013). This review process produces modern, relevant cyber data which is labeled as either legitimate or malicious. High quality labeled cyber data of this type is a scarce commodity in the cyber research community (Sommer & Paxson, 2010).

There have been many attempts to generate cyber data sets for IDS research and development. Tavallaee, Bagheri, Lu, and Ghorbani (2009) and McHugh (2000) provide constructive critiques of the ubiquitous KDD-CUP cyber data set (Stolfo, Wei Fan, Wenke Lee, Prodromidis, & Chan, 2000). Notable issues of the data set include spurious repetitions and biased ML training. Both KDD-CUP and the improved NSL-KDD data set (Tavallaee et al., 2009) are still used in modern NIDS research (Chalé, Bastian, & Weir, 2020; Serinelli, Collen, & Nijdam, 2020). Many other data sets are reviewed by Hindy et al. (2020) and Lashkari, Kadir, Taheri, and Ghorbani (2018). Each manuscript concludes that data sets become deprecated almost as soon as they are published because cyber strategies are constantly adapting to IDS. Further, many data sets lack the statistical information and realness required to train a NIDS. In particular, modern malware oriented at the IoBT does not express its malicious behavior until deployed in a realistic setting (Lashkari et al., 2018). This implies that data sets should be built by collecting data in a real life or a realistic simulated setting. The most skilled hackers understand the behavior that triggers alerts from widely distributed NIDS (Stallings et al., 2012). This implies that NIDS require regular updates.

According to Hindy et al. (2020), NIDS training data sets should be expandable over time and easily accessible to the research community. Data should include realistic background traffic. Larger training sets provide a degree of robustness against hackers attempting to defeat NIDS. The study found approximately 11% of NIDS in their review were trained with simulated data (Hindy et al., 2020). The task of feature selection for cyber data is complicated by the broad range of data types utilized for IDS. Stallings et al. (2012) recounts *Network Security Monitor*, the first IDS to use network flow data. This practice is continued today with network security applications such as SNORT and Zeek (Stallings et al., 2012).

Maxwell, Alhajjar, and Bastian (2019) collect real, labeled cyber data and report the performance of NIDS ML classifiers when they are trained on a variety of engineered features. The U.S. Army Engineer Research and Development Center (ERDC) maintains the Cybersecurity Environment for Detection, Analysis, and Reporting (CEDAR) database of network traffic passing on the Defense Research and Engineering Network (DREN). Maxwell et al. (2019) queried CEDAR and selected a data set of 250,000 cyber alert records which have been labeled by analysts as either normal or malicious. 16 data fields from this set were judged to be potentially important to train ML models for NIDS. Another study introduced a novel technique called attribute ratio, which yielded better classification accuracy than existing statistical approaches to feature engineering (Chae, Jo, Choi, & Park, 2013).

2.3. Data augmentation

In the image domain for computer vision, the lack of diverse training examples may yield high variance machine learning models. That is, validation testing demonstrates an undesirable dependency on the specific training examples used and the final model. Models that suffer from high variance are unable to generalize well to unseen test examples. Techniques such as dropout, batch normalization, transfer learning, pre-training, meta-learning, and data augmentation have been shown to ameliorate variance in models (Shorten & Khoshgoftaar, 2019). Data augmentation, in particular, provides additional data examples with similar, but modified information compared to the baseline examples. Typically, models perform better when they are provided additional high quality data points from the target distribution. The augmented data sets allow machine learning models to generalize patterns more effectively. Basic data augmentation techniques in the image domain include kernel filters, geometric transformations, random erasing, color space transformations, and image mixing. Additionally, deep learning augmentation methods such as adversarial training, neural style transfer, and generative adversarial network perturbations have been used for data augmentation (Shorten & Khoshgoftaar, 2019). Both basic and deep methods for data augmentation have performed well in empirical studies (Goodfellow, Bengio, & Courville, 2016; Shorten & Khoshgoftaar, 2019).

2.4. Data sampling

Data is often procured with suboptimal class distributions for training machine learning models (Kerwin & Bastian, 2020). Data oversampling methods are used to reduce model variance and they have been applied to a variety of domains. Oversampling is especially useful in the case that minority class is not well represented in the training data and in applications where misclassifying the minority classes (false positive & false negative) are more damaging than misclassification of the common classes (Kerwin & Bastian, 2020; Shorten & Khoshgoftaar, 2019). Japkowicz (2000) investigates oversampling the minority class, undersampling the majority class, and a focused oversampling of minority class among points that lie near the decision boundary. While both undersampling and oversampling were demonstrated to improve classification accuracy, there was no additional benefit for the focused oversampling method. Recognition is another approach sought to perform positive class recognition on the majority class if the training set has a class imbalance. The recognition approach is only useful, however, for the majority class; nothing is learned on the minority classes (Japkowicz, 2000). Relevant background on single class recognition and anomaly detection can be found in Chandola, Banerjee, and Kumar (2009), Hindy et al. (2020), Japkowicz, Myers, Gluck, et al. (1995). Kerwin and Bastian (2020) demonstrate how Synthetic Minority Over-sampling Technique (SMOTE), (Chawla, Bowyer, Hall, & Kegelmeyer, 2002), and its contemporary variants use k-nearest neighbors to generate synthetic labeled points that improve the decision boundaries of ML classifiers. The synthetic examples generated by SMOTE are convex combinations of training data and, therefore, cannot extrapolate beyond the convex hull of neighboring points. SMOTE also does not enforce any conditional relationship for categorical variables. Kerwin and Bastian (2020) expand on these methods by incorporating a stack ensemble to learn the best combination of classifier-sampler pairing for improved classification performance on highly imbalanced data sets. They find that the stack ensemble does provide good classification on a clean, labeled data set but it is yet to be shown why certain pairings perform well or how they will perform outside the laboratory setting. Lim, Loo, Tran, Cheung, Roig, and Elovici (2018) builds on the concepts of SMOTE oversampling but performs the sampling within the latent space of an autoencoder. These sampled points in the latent space are then filtered to return only those within a specified distance to the decision boundary. Still within the latent space, the k-nearest neighbors to the sampled points are returned. The neighbors are decoded to the $\mathcal{X}$-space where they are recorded as samples. Generative models go beyond sampling techniques to model the entire joint distribution of data so realistic examples can be synthetically generated from the complex distribution.

2.5. Generative methods

Although previous academic studies (Alhajjar, Maxwell, & Bastian, 2021; Chalé et al., 2020; Devine & Bastian, 2021; Hindy et al., 2020; Tavallaee et al., 2009) have demonstrated the great potential of using ML techniques for network intrusion detection, they typically use low quality, outdated data. Adapting models from the laboratory to the real world is especially challenging due to the lack of real world labeled training data (Sommer & Paxson, 2010). The conventional approach to machine learning is to train a model that models a conditional relationship of class y given an input vector $\mathbf{x}$, but it does not model the overarching joint distribution of a population. These types of models are known as discriminative models and have been utilized in the field of statistical learning for decades. A model that does capture the joint distribution of training data is necessary for synthetic data generation. These models are collectively known as generative methods. A significant volume of recent work in generative modeling is in the field of machine learning (Goodfellow et al., 2016; Jebara, 2012), though other generative methods are rooted computational Bayesian statistics (Gelman, Carlin, Stern, Dunson, Vehtari, & Rubin, 2013; Hastings, 1970).

Generative models are appealing in order to achieve high classification performance (Japkowicz et al., 1995; Jebara, 2012) and further, to generate desperately needed training data for discriminative models, particularly deep models that require large data sets (Chalé & Bastian, 2021). The computational Bayesian approaches of generative methods have been a concentrated area of research since the early contributions of Hastings (1970), Metropolis, Rosenbluth, Rosenbluth, Teller, and Teller (1953). Markov Chain Monte Carlo (MCMC) generative methods explore the model's parameter space $\boldsymbol{\theta}$ for values that maximize the correctness of the posterior distribution $p(\boldsymbol{\theta}|y)$ with respect to a target distribution. With each sampling of $\boldsymbol{\theta}$, the correctness of the current posterior is assessed and the step is either accepted or rejected. This process is repeated until a stopping criteria is satisfied (Gelman et al., 2013; Yildirim, 2012). MCMC techniques possess the Markovian property which states convergence to a stationary distribution is theoretically attainable regardless of the initialized values $\boldsymbol{\theta}_0$. Gibbs Sampling is the most basic form of MCMC while Metropolis–Hastings MCMC is a more advanced variant that uses probabilistic acceptance of steps. This strategy of probabilistic acceptance provides the Metropolis–Hastings algorithm the property of *detailed balance* (Kass, Carlin, Gelman, & Neal, 1998). With detailed balance satisfied, the algorithm is guaranteed to obtain a posterior distribution matching the target distribution in a finite number of steps. Research suggests the Metropolis–Hastings algorithm is preferred for many systems because it is less likely to get stuck in local minima (Yildirim, 2012).

Chalé and Bastian (2021) find that in practice, the Metropolis–Hastings algorithm is not computationally tractable due to the low chance of accepting each step when there are many parameters $\boldsymbol{\theta}$, such as when modeling high dimensional cyber data. Chalé and Bastian (2021) suggest further work in generative techniques for synthetic cyber data should focus on generative machine learning. The Boltzmann Machine is a fundamental achievement in generative machine learning used to approximate the distribution of observed variables by learning a model's latent variables (Hinton, Sejnowski, & Ackley, 1984). For discrete distributions, the Boltzmann Machine is a universal approximator of data (Goodfellow et al., 2016). Smolensky (1986) drew inspiration from cognitive science to improve the structure of the Boltzmann Machine. Denoted as the *Restricted Boltzmann Machine*,

Table 1
Data fields and feature engineering technique.

Data field	Example	Technique
connection	open, close, none	Label encoded
id_orig_h_cc	US	Label encoded
id_resp_h_cc	US	Label encoded
severity	H, M, L	Label encoded
method	GET, POST	Label encoded
request_body_len	10	minimax
response_body_len	10	minimax
hp	0000000000000000	16 bit vector
id_orig_h_org	external	Label encoded
id_resp_h_org	internal	Label encoded
host	my.connection.edu	Label encoded
status	normal, bad	One hot encoded

this model is bipartite, with no connections between nodes on the observed layer or between nodes in the latent layer. This new structure captures complex relationships between variables and can be stacked for additional capacity. The conditional distributions of variables can be derived and sampled from the trained Restricted Boltzmann Machine. Variational autoencoders are a generative deep learning technique that uses gradient methods to learn parameters (Goodfellow et al., 2016; Kingma & Welling, 2014). Parameters are trained by maximizing the variational lower bound of data in the training set. New data examples can be generated from a trained variational autoencoder by drawing a random sample $\mathbf{z}$ from the distribution $p_{model}(\mathbf{z})$ and then passing $\mathbf{z}$ through the layers of the autoencoder g. The generator produces output sample $\mathbf{x}$ taken from $p_{model}(\mathbf{x}; g(\mathbf{z})) = p_{model}(\mathbf{x}|\mathbf{z})$ (Goodfellow et al., 2016). Another deep generative method, generative adversarial networks (GANS), provide a breakthrough for synthetic data generation (Goodfellow, Pouget-Abadie, Mirza, Xu, Warde-Farley, Ozair, Courville, & Bengio, 2014). As the name suggests, the architecture is comprised of two neural networks competing in an iterative game. A generator network, g, generates synthetic data $\mathbf{x} = g(\mathbf{z}; \boldsymbol{\theta}^{(g)})$, where $\boldsymbol{\theta}^{(g)}$ are the learned parameters of the generator. The other network is discriminator, d, which learns to classify synthetic examples x as real or synthetic via the operation $d(\mathbf{x}; \boldsymbol{\theta}^{(d)})$, where $\boldsymbol{\theta}^{(d)}$ are the learned parameters of the discriminator. The competing networks are simultaneously trained via gradient backpropagation. Initially, the generated examples are poor quality and the discriminator easily detects synthetic examples. Through continued training, the generator produces higher quality synthetic examples with likeness to the real data. Even as the discriminator becomes a better critic of synthetic data, the well trained generator can produce synthetic data that is indistinguishable from the real data. The fully trained generator is then used to draw synthetic examples from the joint distribution of the real data.

Chalé and Bastian (2021) proposed using generative methods to expand on the limited supply of high quality cyber data. Markov Chain Monte Carlo generated realistic continuous features but proved ineffective with high dimensional data sets (Chalé & Bastian, 2021). The conditional tabular generative adversarial network (CTGAN) and tabular variational autoencoder (TVAE) demonstrated capability to synthesize continuous data in Chalé and Bastian (2021). CTGAN and TVAE demonstrated capability to generate synthetic data of both continuous *and* discrete high dimensional data in Xu, Skoularidou, Cuesta-Infante, and Veeramachaneni (2019); however, these studies used the obsolete KDD-CUP and NSL-KDD data sets which are not suitable to train a modern NIDS. Our manuscript explores whether high quality, synthetic, data generated from generative models can serve as a proxy for real data when training ML-based NIDS.

3. Methodology

The methodology applied in this research addresses three key issues that plague current cyber data sets: data quality, data currency, and metric-based evaluation.

(1) Data quality: The realness, relevancy, and recentness of baseline data used in this approach can be guaranteed because it is obtained from actual cyber traffic on the DREN. The baseline data contains large quantities of background data packets as well as alert data. The Zeek NIDS in DREN uses specialized scripts and algorithms to flag alerts and severity levels in potentially malicious traffic. A computer incident response team (CIRT) adjudicates the alert packets as normal or bad. Although the alert threshold is set conservatively low, it is impossible to make any conclusions about false negatives (i.e., malicious packets that were not flagged for human review). Therefore, the label assigned by the human analyst is considered truth. Synthetic data generation seeks to provide additional data examples of the same statistical distribution, and therefore same high quality as the real data.

(2) Data currency: The cyber data queried from DREN can be updated and expanded iteratively. The network's CIRT provides continuous monitoring and new queries can produce new cyber records as they become available. Additional data fields can be incorporated into updated data sets as well, assuming they can be captured by Zeek. Synthetic data generation is used to increase the quantity of labeled examples. This is especially important when zero day attacks have been detected, but there are not enough examples to train and evaluate a ML classifier.

(3) Metrics: The realness of synthetic NIDS training data can be assessed with descriptive statistical metrics and by comparing the performance of standard classifiers on the generated data to the real labeled data (Chalé & Bastian, 2021; Xu et al., 2019). These approaches are prevalent in the literature and should become the expectation in related cybersecurity research. The inverted Kolmogorov–Smirnov (KS) d statistic (Patki, Wedge, & Veeramachaneni, 2016) and classification recall (Chalé et al., 2020) work especially well to convey an overall quality of synthetic data. The inverted D statistic is provided by the Synthetic Data Vault package (Patki et al., 2016). It is calculated as the average of 1 minus the d statistic for each column in a pair of data sets. As noted in Eq. (1), the D statistic is calculated as the greatest observed difference between two distributions across all x (Law & Kelton, 2007; Massey Jr, 1951). The d statistic is limited however in that it does not capture covariance in multivariate distributions.

$$d_N^{(i)} = \max\{|F_0^{(i)}(x) - S_N^{(i)}(x)|\} \tag{1}$$

In Eq. (1), $d^{(i)}$ denotes the Kolmogorov–Smirnov d statistic specifically for column i in the data set. $F_0^{(i)}(x)$ denotes the observed distribution of real baseline data in column i. $S_N^{(i)}(x)$ denotes the observed distribution of synthetic data in column i. N is the number of synthetic observations in the sample. Critical values of d, with N observations and significance α are given by Massey Jr (1951).

3.1. Collect baseline cyber data

All DREN traffic is monitored with Zeek. Zeek reports low, medium, and high confidence alerts for suspicious traffic detected by its community sourced scripts. Packets flagged with alerts are summarized with 73 features and stored on the CEDAR database. These records are then audited by the CIRT who adjudicates them as normal or malicious. The ERDC maintains these labeled feature vectors on CEDAR and approved researchers can query the records using the High Performance Computing Architecture for Cyber Situational Awareness (HACSAW) API. ERDC prohibits HACSAW data from being removed from the DREN network, therefore, all data analysis must be performed on the DREN.

This research follows the data collection method of Maxwell et al. (2019) in which HACSAW is queried for real, labeled, logs of HTTP cyber traffic on the DREN. For the first research question in this study, the 250,000 data examples from Maxwell et al. (2019) were obtained and reused. For the second research question, however, a greater number of examples were required, so a new data set was queried and pre-processed.

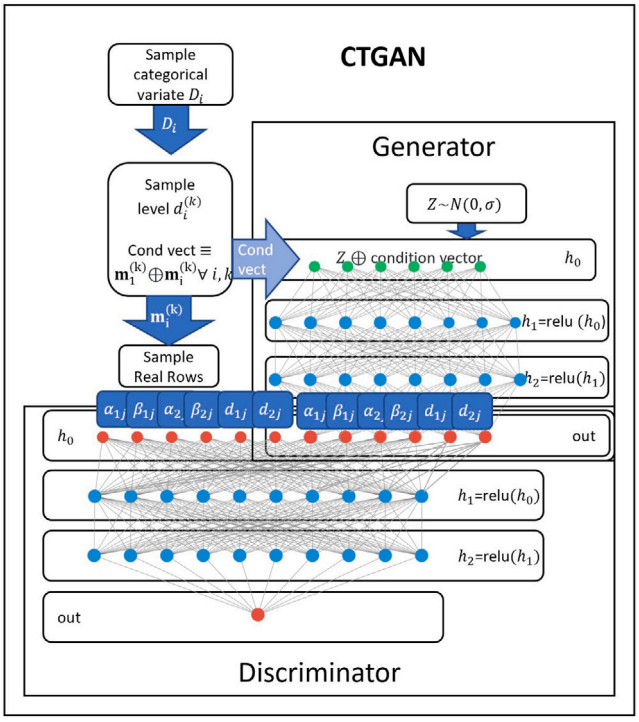


Fig. 1. CTGAN includes a generator network that is input to a discriminator network, as proposed by Xu et al. (2019).

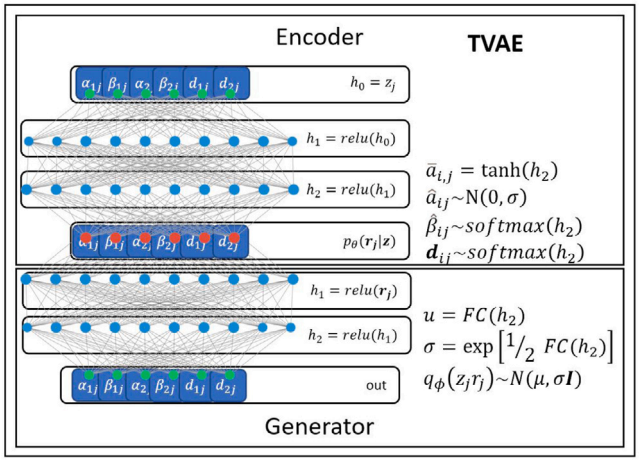
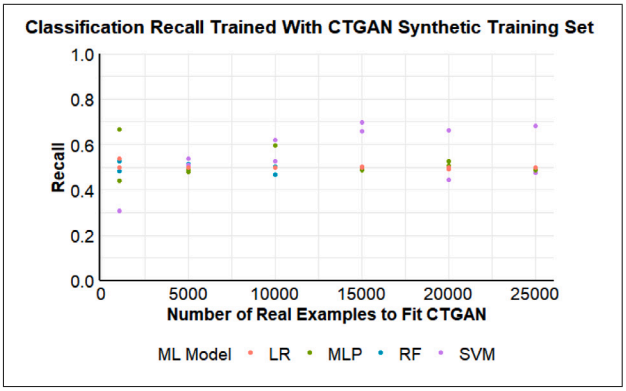
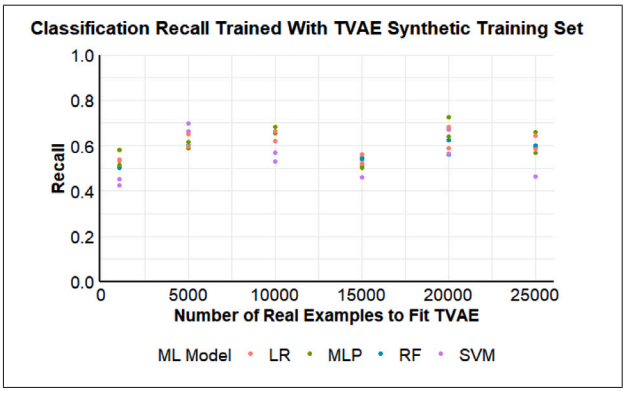


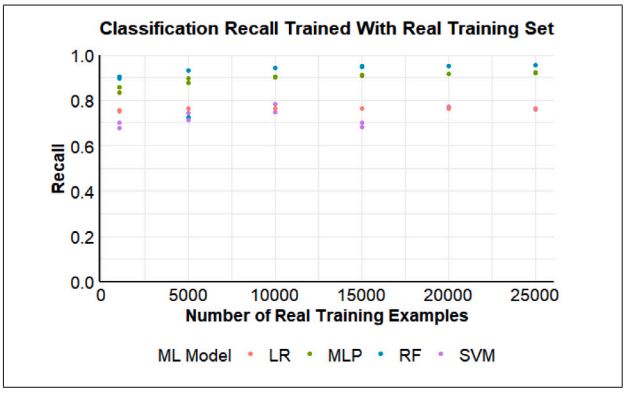
Fig. 2. TVAE is a tabular variational autoencoder with an encoding network and a decoding/generator network, proposed by Xu et al. (2019).



(a) ML models trained with CTGAN generated cyber data



(b) ML models trained with TVAE generated cyber data



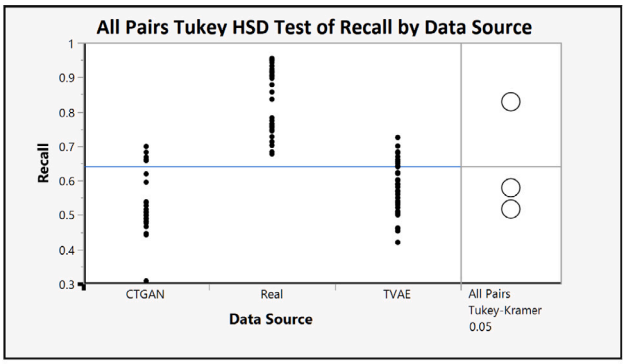
(c) ML models trained with real cyber data

Fig. 3. Classification recall of models trained with real or synthetic data.

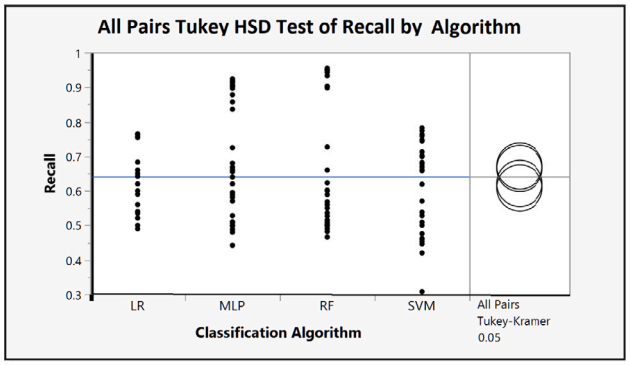
Details on HTTP protocol and the contents of the Zeek HTTP logs are outlined by the Zeek documentation (Siwek, 2021). This study also references the feature engineering method by Maxwell et al. (2019), which provides guidance on optimizing features for machine learning with cyber data. Table 1 outlines which engineered features were derived from the HTTP logs and used for this study.

3.2. Generate synthetic cyber data

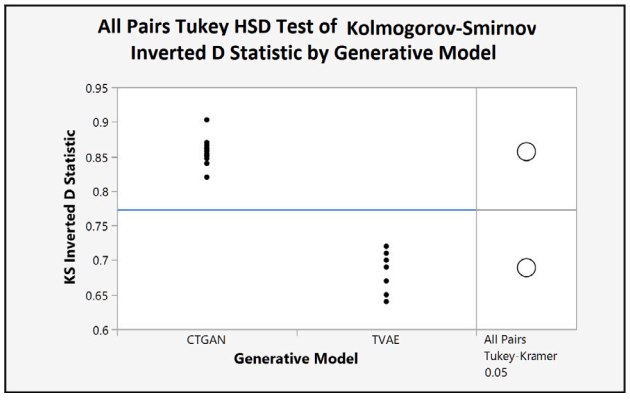
Recent research demonstrates that generating synthetic, tabular data has unique challenges that do not exist in other domains such as synthetic image generation (Chalé & Bastian, 2021; Xu et al., 2019). Tabular data contains variates that are continuous, or discrete, and in some cases a particular variate may contain both types of observations. Although image pixels may follow a roughly Gaussian distribution, the joint distribution of data tables do not follow any known distribution. Data tables tend to be multimodal, and existing generative methods such as a GAN perform poorly in multimodal domains. Discrete variates are typically one hot encoded, which is practical to model for univariate data. It is unclear, however, how to model joint distributions with discrete and continuous variates. Further, violations to one hot sparsity in the generated data would be easily detectable with simple rules. Finally, the categorical variates may predominately express major classes, and only express a few instances of the minor classes. Generative models may fail to capture the non-zero probability of minor modes despite converging to an optimal model (Xu et al., 2019).



(a) Plot of recall of the real, TVAE, and CTGAN data on left, and connected groups with Tukey's HSD test of equivalent means on right.



(b) Plot of recall by machine learning classifier on the left, and connected groups with Tukey's HSD test of equivalent means on right.



(c) Plot of Kolmogorov-Smirnov inverted D statistic by generative model on left, and connected groups with Tukey's HSD test of equivalent means on right.

Fig. 4. Significance of treatment groups on classification recall of models trained with real or synthetic data.

Previous results from Chalé and Bastian (2021) indicated CTGAN, TVAE, and Markov-Chain Monte-Carlo (MCMC) with Metropolis Hastings algorithm were potential choices for modeling the joint distribution of cyber data. Our follow up trials demonstrated that only CTGAN and TVAE could be scaled for high dimensional data sets. Exploratory trials of MCMC were implemented in pymc3 (Salvatier, Wiecki, & Fonnesbeck, 2016) using Python 3.8. We found that the multivariate Gaussian mixture model (MVGMM) fit with the Metropolis–Hastings

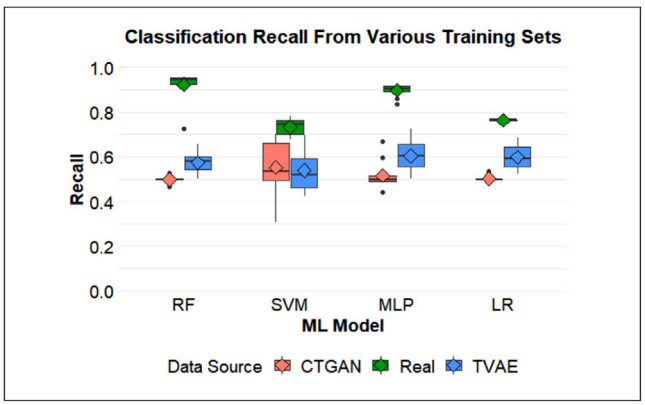


Fig. 5. Classification recall by ML algorithm and data set.

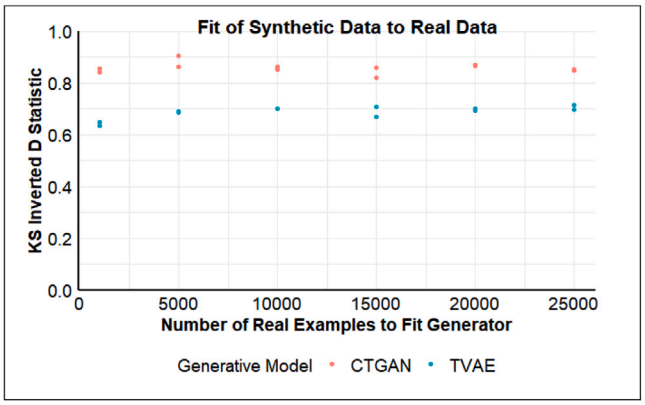


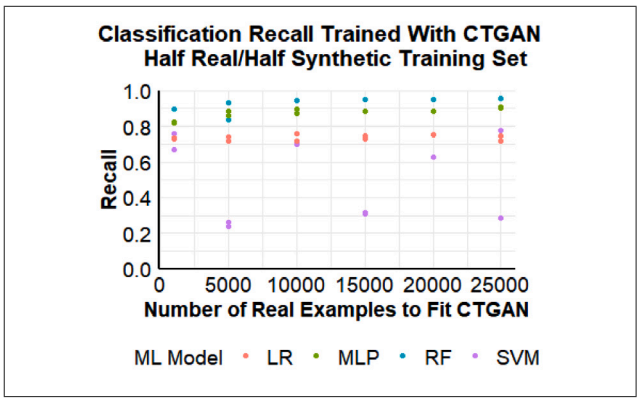
Fig. 6. Similarity between real and synthetic data sets.

MCMC algorithm in Chalé and Bastian (2021) could not be adapted for more than two continuous variables, regardless of posterior distribution and hyperparameter settings. Likewise, the MVGMM fit with the No-U-Turn MCMC algorithm in pymc3 did not converge for data sets greater that two continuous variables. Therefore, CTGAN and TVAE are selected as models to synthetically generate cyber data, with details of these two generative ML methods to follow.

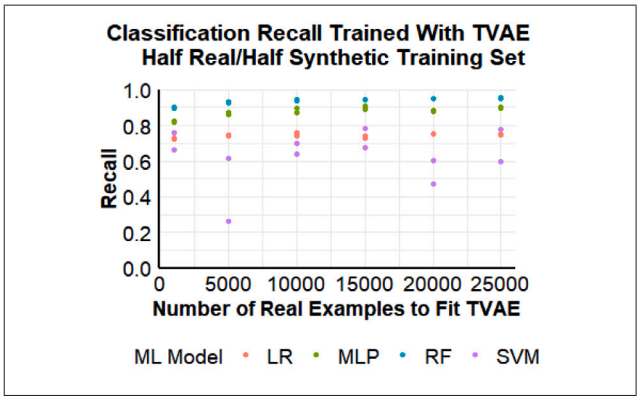
3.2.1. *Generative adversarial networks*

Generative adversarial networks (GANs) are a machine learning framework for building generative models introduced by Goodfellow et al. (2014) and updated in Goodfellow, Pouget-Abadie, Mirza, Xu, Warde-Farley, Ozair, Courville, and Bengio (2020) with recent insight. GANs are built using an adversarial process. On one side, a multi-layer perceptron generates data, initially with random weights, while on the other side, a discriminative MLP detects whether or not the generated data matches the distribution of a target set. Initially, the discriminator can easily distinguish generated data examples from true examples, but the generator improves its model over time. Simultaneously, the discriminator learns on its mistakes and improves its model. Eventually, the generator produces examples that match the original distribution very well, and the discriminator cannot detect a difference in the generated examples (Goodfellow et al., 2014). Most GANs today are based on the deep conventional GAN architecture (Goodfellow, 2017) and used for image generation (Goodfellow et al., 2014). As alluded to in Section 3.2, creating a GAN for tabular data is a more nuanced task.

A GAN capable of modeling tabular data has been constructed (Xu et al., 2019). The model, CTGAN, is designed to address the shortcomings of fitting traditional GANs to tabular data. The model addresses mode collapse of minor discrete classes by conditioning the training on



(a) ML models trained with CTGAN generated cyber data



(b) ML models trained with TVAE generated cyber data

Fig. 7. Classification recall of models trained with a mixture of real and synthetic data.

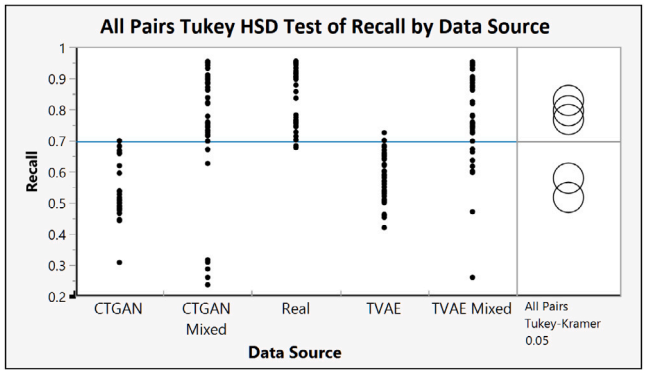
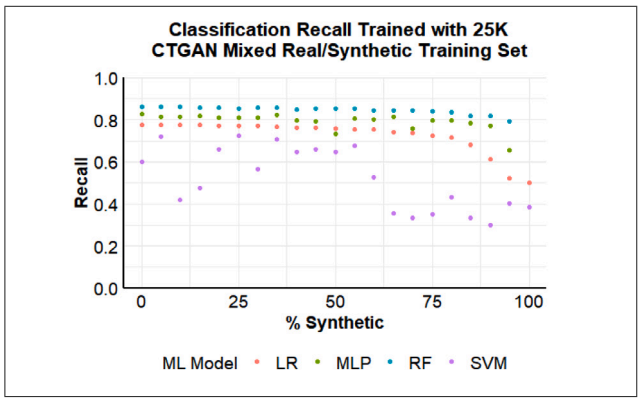


Fig. 8. Plot of recall of the real, synthetic, and 50/50 synthetic and real data on the left, and connected groups with Tukey's HSD test of equivalent means on right.

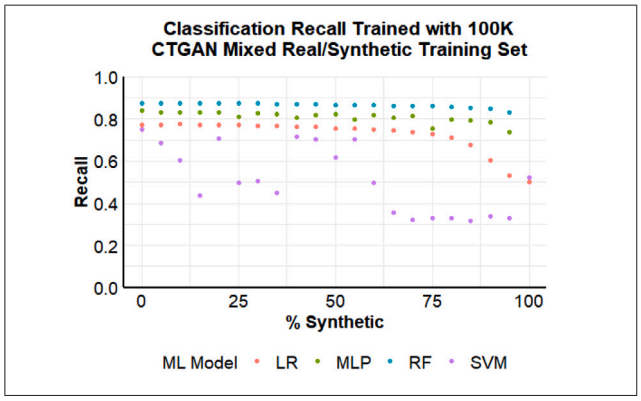
Table 2
Classification models are trained with data sets where the percent synthetic data is varied.

Trial	1	2	3	4	5	6	7	8	9	10	11
Real (%)	100	95	90	85	80	75	70	65	60	55	50
Synthetic (%)	0	5	10	15	20	25	30	35	40	45	50
Trial	12	13	14	15	16	17	18	19	20	21	–
Real (%)	45	40	35	30	25	20	15	10	5	0	–
Synthetic (%)	55	60	65	70	75	80	85	90	95	100	–

each discrete mode. A condition generator is used to sample discrete classes and represent as a one-hot condition vector. The condition



(a) ML models trained with 25,000 examples of real and CTGAN synthetic data



(b) ML models trained with 100,000 examples of real and CTGAN synthetic data

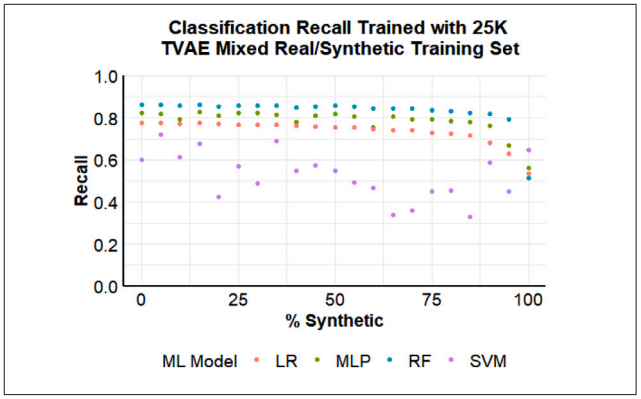
Fig. 9. Classification recall of models where percent CTGAN synthetic data is varied.

Table 3
Connection table of recall by data source using Tukey HSD all pairs test with a confidence of 0.95.

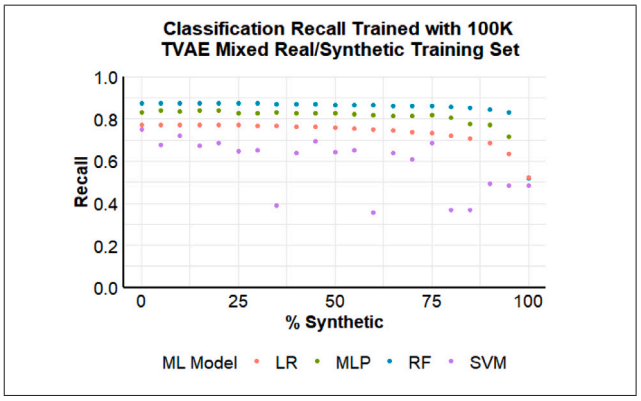
Level	Group	Mean recall
Real	A	0.82887500
TVAE Mixed	A	0.79562500
CTGAN Mixed	A	0.76743750
TVAE	B	0.57883333
CTGAN	B	0.51697917

vector representation of discrete conditions is input to the CTGAN generator. This input motivates the generator to learn relationships between the condition and a realistic synthetic output for all discrete categories. CTGAN also detects modes within continuous variables and determines mode membership for each example using a Guassian variational model. Mode specific normalization facilitates learning for data examples attributed to minor modes. This information is also input to the generator. Collectively, these features of GAN address the challenges of modeling tabular data (Xu et al., 2019).

CTGAN is represented visually in Fig. 1. The network represents variables with a special encoding structure to capture the conditional relationships of variables and to prevent mode collapse. The row containing both discrete and continuous variables is represented by $\mathbf{r}_j$. $\mathbf{r}_j$ contains α_{ij}, a continuous value within a mode for variable i and data example j; β_{ij}, a one hot vector indicating mode membership for variable i; and $\mathbf{d}_{ij}$ a vector specifying the discrete levels active for variable i. Mask vectors $\mathbf{m}_i^{(k)}$ are formed to encode the active discrete settings for all combinations of variables and settings where $\mathbf{m}_i^{(k)} = 1$ if



(a) ML models trained with 25,000 examples of real and TVAE synthetic data



(b) ML models trained with 100,000 examples of real and TVAE synthetic data

Fig. 10. Classification recall of models where percent TVAE synthetic data is varied.

setting k is active for variable i. The concatenation of all mask vectors for a data example is known as the condition vector.

The input layer of the CTGAN generator includes the condition vector and a vector of Gaussian noise. Two hidden layers of 256 nodes are fully connected with batch normalization, dropout and leakyrelu activation functions. The output layers are also fully connected. The continuous output are assigned with a tanh activation function while the mode indicator vector and discrete values are assigned with a gumbelsoftmax activation function. The discriminator is a MLP with two hidden layers, and leakyrelu activation functions, that discerns whether the generated row vectors are drawn from the same population as the sampled row vectors. The parameters in the generator and discriminator are simultaneously trained with Wasserstein loss via the Adam optimizer.

3.2.2. Autoencoders

Autoencoders are a deep learning framework that learns a function that summarizes input data in a low dimensional latent space and reproduces the data in the original space with minimal error. This is done by constructing a neural network with two sequence of layers. The first sequence of layers $\mathbf{h} = f(\mathbf{x})$ compresses the input to the latent space, and another sequence of layers $g(f(\mathbf{x})) = \mathbf{x}$ generates realistic data from the latent encoding (Goodfellow et al., 2016).

Variational autoencoders are more advanced than standard autoencoders because they are trained to maximize the variational lower bound, $\mathcal{L}(q)$, of training data (Kingma & Welling, 2013). This choice of loss function motivates the model to place high probability across

Table 4
Classification recall of machine learning models trained with CTGAN synthetic data.

Classifier	Real examples	Recall
RF	1,000	0.527
RF	5,000	0.501
RF	10,000	0.501
RF	15,000	0.5
RF	20,000	0.49
RF	25,000	0.499
RF	1,000	0.482
RF	5,000	0.516
RF	10,000	0.466
RF	15,000	0.498
RF	20,000	0.5
RF	25,000	0.5
SVM	1,000	0.5
SVM	5,000	0.538
SVM	10,000	0.619
SVM	15,000	0.699
SVM	20,000	0.661
SVM	25,000	0.476
SVM	1,000	0.308
SVM	5,000	0.509
SVM	10,000	0.528
SVM	15,000	0.658
SVM	20,000	0.446
SVM	25,000	0.682
MLP	1,000	0.668
MLP	5,000	0.489
MLP	10,000	0.5
MLP	15,000	0.498
MLP	20,000	0.528
MLP	25,000	0.488
MLP	1,000	0.442
MLP	5,000	0.48
MLP	10,000	0.595
MLP	15,000	0.489
MLP	20,000	0.507
MLP	25,000	0.5
LR	1,000	0.5
LR	5,000	0.5
LR	10,000	0.5
LR	15,000	0.501
LR	20,000	0.49
LR	25,000	0.5
LR	1,000	0.536
LR	5,000	0.5
LR	10,000	0.5
LR	15,000	0.5
LR	20,000	0.5
LR	25,000	0.5

many values of the latent variables $\mathbf{z}$ that could feasibly reconstruct $\mathbf{x}$ (Goodfellow et al., 2016).

During training, we take the following perspective of the model. The approximate inference network $q(\boldsymbol{z}|\boldsymbol{x})$ is the encoder that generates $\boldsymbol{z}$; and the decoder is the distribution $p_{model}(\boldsymbol{x}|\boldsymbol{z})$ from which the reconstructed output is drawn (Goodfellow et al., 2016). The variational lower bound, $\mathcal{L}(q)$ for the sample $\mathbf{x}$ is maximized as shown in Eq. (2).

$$\mathcal{L}(q) = \mathbb{E}_{\boldsymbol{z}q(\boldsymbol{z}|\boldsymbol{x})} \log p_{\text{model}}(\boldsymbol{z}, \boldsymbol{x}) + \mathcal{H}(q(\boldsymbol{z}|\boldsymbol{x})) \tag{2}$$

TVAE is a variational autoencoder designed specifically to model and generate tabular data. The autoencoder uses the same variable encoding as CTGAN to address mode collapse and capture conditional relationships. TVAE encoder contains two hidden layers with the relu activation function. An output layer contains tanh activation functions for continuous variables, α_{ij} and softmax activation functions for discrete variables, β_{ij} and d_{ij}.

The network produces one hot encoded values for discrete variables β_{ij} and $\mathbf{d_{ij}}$ in $\mathbf{r_j}$. For α_{ij}, the continuous variables, the network captures the mean and standard deviation. A generator model with two hidden layers then uses the learned distribution to generate normally distributed values for α, hence, fully defining $\mathbf{r}_j$ (see Fig. 2).

Table 5
Classification recall of machine learning models trained with TVAE synthetic data.

Classifier	Real examples	Recall
RF	1,000	0.504
RF	5,000	0.589
RF	10,000	0.568
RF	15,000	0.536
RF	20,000	0.623
RF	25,000	0.602
RF	1,000	0.51
RF	5,000	0.60
RF	10,000	0.66
RF	15,000	0.55
RF	20,000	0.56
RF	25,000	0.59
SVM	1,000	0.453
SVM	5,000	0.661
SVM	10,000	0.529
SVM	15,000	0.51
SVM	20,000	0.669
SVM	25,000	0.462
SVM	1,000	0.42
SVM	5,000	0.70
SVM	10,000	0.57
SVM	15,000	0.46
SVM	20,000	0.57
SVM	25,000	0.46
MLP	1,000	0.581
MLP	5,000	0.588
MLP	10,000	0.655
MLP	15,000	0.508
MLP	20,000	0.725
MLP	25,000	0.66
MLP	1,000	0.51
MLP	5,000	0.62
MLP	10,000	0.68
MLP	15,000	0.50
MLP	20,000	0.64
MLP	25,000	0.57
LR	1,000	0.535
LR	5,000	0.65
LR	10,000	0.62
LR	15,000	0.521
LR	20,000	0.683
LR	25,000	0.642
LR	1,000	0.54
LR	5,000	0.60
LR	10,000	0.66
LR	15,000	0.56
LR	20,000	0.59
LR	25,000	0.59

Table 6
Classification recall of machine learning models trained with real data.

Classifier	Examples	Recall
RF	1000	0.903
RF	5000	0.727
RF	10000	0.943
RF	15000	0.95
RF	20000	0.952
RF	25000	0.955
RF	1000	0.898
RF	5000	0.933
RF	10000	0.943
RF	15000	0.948
RF	20000	0.952
RF	25000	0.955
SVM	1000	0.677
SVM	5000	0.744
SVM	10000	0.747
SVM	15000	0.703
SVM	20000	0.763
SVM	25000	0.759
SVM	1000	0.702
SVM	5000	0.713
SVM	10000	0.782
SVM	15000	0.683
SVM	20000	0.774
SVM	25000	0.759
MLP	1000	0.836
MLP	5000	0.897
MLP	10000	0.904
MLP	15000	0.914
MLP	20000	0.916
MLP	25000	0.919
MLP	1000	0.857
MLP	5000	0.878
MLP	10000	0.902
MLP	15000	0.907
MLP	20000	0.915
MLP	25000	0.924
LR	1000	0.754
LR	5000	0.764
LR	10000	0.764
LR	15000	0.763
LR	20000	0.765
LR	25000	0.764
LR	1000	0.757
LR	5000	0.764
LR	10000	0.766
LR	15000	0.763
LR	20000	0.764
LR	25000	0.764

3.3. Machine learning classifiers

Humans posses strong ability to interpret information and draw conclusions. In particular, humans can learn patterns allowing them to classify things into groups. Similarly, statistical models can be trained with large training sets to label unlabeled data examples. These models are machine learning classifiers, meaning the algorithm has learned the model from the data. A training data set includes vectors of observations $(\mathbf{x}_n, y_n)$ where $\mathbf{x}_n$ are the indicators and y_n is the label. An unlabeled data set includes only the indicator variables $\mathbf{x}_{n'}$, but a trained model can assign the label $y_{n'}$. Unlike regression models, which estimate a continuous response y, classification models assign a discrete class (James, Witten, Hastie, & Tibshirani, 2013). Most machine learning techniques require the data is prepared with several steps of pre-processing. Feature engineering is one such step (Chollet, 2021). In this research, we reference the successful engineered features for cyber data presented by Maxwell et al. (2019). There is a growing list of machine learning methods that perform classification. Several of the most pervasive methods have been selected for this study.

3.3.1. Logistic regression

Linear regression is a statistical model that captures the relationship between indicator variables and a continuous response variable (Montgomery, Peck, & Vining, 2012). The linear regression model takes the form of Eq. (3) where $\mathbf{x}$ is a vector of predictor variables, $\hat{\beta}$ is the least squares estimator of the model parameters, and $\hat{y}$ is the prediction of the response variable.

$$\hat{y} = \mathbf{x}'\hat{\beta} \tag{3}$$

The model, however does not perform well for binary or categorical outputs. In the binary case, a trained model may output a response anywhere between 0 and 1, or well outside that range. These responses have no statistical interpretation, therefore, linear regression is ill suited for the binary classification problem (James et al., 2013). Cox (1958) argues that the preferred form of regression for binary classification is given by Eq. (4).

$$pr(Y_i = 1) = e^{\alpha+\beta_i X}/(1 + e^{\alpha+\beta_i X}) \tag{4}$$

with indicator variable x, parameters β_0 and β_1 (Cox, 1958). The logistic regression formula, Eq. (4) yields a response between 0 and 1 for all input.

Table 7
Inverted Kolmogorov Smirnov D-Statistic of data generated by TVAE and CTGAN, varying the quantity of real data to fit the generators.

Data source	Real examples	KS-D
CTGAN	1,000	0.854
CTGAN	5,000	0.862
CTGAN	10,000	0.852
CTGAN	15,000	0.858
CTGAN	20,000	0.866
CTGAN	25,000	0.851
CTGAN	1,000	0.840
CTGAN	5,000	0.903
CTGAN	10,000	0.861
CTGAN	15,000	0.820
CTGAN	20,000	0.870
CTGAN	25,000	0.847
TVAE	1,000	0.65
TVAE	5,000	0.69
TVAE	10,000	0.70
TVAE	15,000	0.67
TVAE	20,000	0.70
TVAE	25,000	0.70
TVAE	1,000	0.64
TVAE	5,000	0.69
TVAE	10,000	0.70
TVAE	15,000	0.71
TVAE	20,000	0.70
TVAE	25,000	0.72

By annotating $p(X) = pr(Y = 1|X)$ we solve for the odds function, Eq. (5),

$$\frac{p(X)}{1 - P(X)} = e^{\beta_0 + \beta_1 X} \tag{5}$$

which provides the ratio of the conditional probability $pr(Y = 1|X)$ and its compliment, given the estimated model.

The log-odds function, given in Eq. (6)

$$log\left(\frac{p(X)}{1 - P(X)}\right) = \beta_0 + \beta_1 X \tag{6}$$

has a linear relationship to X. That is, changing the parameter X by a unit, affects the log odds by β_1. β_0 and β_1 are now conveniently estimated by the method of maximum likelihood. Once the parameters are estimated, the logistic regression model is given by the probability distribution $p(X)$ shown in Eq. (4) (James et al., 2013).

This research implements LogisticRegression in scikit-learn with the liblinear solver, which is preferred for smaller data sets. All other parameters are set to default. Notably, the scikit-learn function LogisticRegression uses L2 regularization with a weight of 10 to remove less important features from the model.

3.3.2. Support vector machine

The support vector machine was initially presented by Boser, Guyon, and Vapnik (1992) and expanded for inseparable data sets in Cortes and Vapnik (1995). It works by using a kernel to project the training data into a higher dimension. In the higher dimensional representation, the algorithm searches for the hyperplane that optimally separates data points by class, and maintains a large margin between classes, and minimal violations. Eq. (7) is the loss function for support vector machines which is used to find $\mathbf{w}$, the optimal separating hyperplane. The first term, $\frac{1}{2}\left\|\mathbf{w}\right\|^2$ is inverse proportional to the separating region between classes. The second term, $C\left(\sum_{i=1}^{N} \xi^{(i)}\right)$ sums the magnitude of $\xi^{(i)}$, the classification error for data examples $x^{(i)}$, which lie on the wrong side of the margin boundary. C is a user defined hyperparameter to balance the importance of the two terms. The constraints in Eqs. (8) and (9) bind ξ to the distance of violation to the margin region. $y^{(i)}$ is the true class label and w_0 is the intercept term of the separating hyperplane (Cortes & Vapnik, 1995).

Table 8
Classification recall of machine learning models trained with half CTGAN synthetic data and half real data.

Classifier	Real examples	Recall
RF	1000	0.899
RF	5000	0.838
RF	10000	0.942
RF	15000	0.95
RF	20000	0.951
RF	25000	0.954
RF	1000	0.899
RF	5000	0.932
RF	10000	0.943
RF	15000	0.949
RF	20000	0.951
RF	25000	0.954
SVM	1000	0.671
SVM	5000	0.236
SVM	10000	0.758
SVM	15000	0.316
SVM	20000	0.626
SVM	25000	0.287
SVM	1000	0.759
SVM	5000	0.26
SVM	10000	0.698
SVM	15000	0.309
SVM	20000	0.626
SVM	25000	0.778
MLP	1000	0.822
MLP	5000	0.885
MLP	10000	0.895
MLP	15000	0.887
MLP	20000	0.883
MLP	25000	0.911
MLP	1000	0.819
MLP	5000	0.862
MLP	10000	0.871
MLP	15000	0.887
MLP	20000	0.883
MLP	25000	0.905
LR	1000	0.733
LR	5000	0.719
LR	10000	0.715
LR	15000	0.747
LR	20000	0.753
LR	25000	0.716
LR	1000	0.731
LR	5000	0.743
LR	10000	0.757
LR	15000	0.726
LR	20000	0.753
LR	25000	0.748

$$\mathcal{L} = arg\ \min_{\mathbf{w}} : \frac{1}{2}\left\|\mathbf{w}\right\|^2 + C\left(\sum_{i=1}^{N} \xi^{(i)}\right) \tag{7}$$

$$S.T. \qquad \xi^{(i)} \geq 0\ \forall i \tag{8}$$

$$y^{(i)}(w_0 + \mathbf{w}^{\top}\mathbf{x}^{(i)}) \geq 1 - \xi^{(i)}\ \forall\ i \tag{9}$$

The NuSVC function in scikit-learn uses a dual formulation to exploit computational efficiencies. By default, this function uses the radial basis function kernel, nu = 0.5, an adaptive formula for gamma, and a kernel cache of 200 mB (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot, & Duchesnay, 2011). In this study, nu is set to 0.2 and the cache is increased to 500 mB.

3.3.3. Multi-layer perceptron

A multi-layer perceptron (MLP) is a mathematical model translating an input vector to a response through one or more layers of intermediary functions. The model was developed over several decades

Table 9
Classification recall of machine learning models trained with half TVAE synthetic data and half real data.

Classifier	Real examples	Recall
RF	1000	0.896
RF	5000	0.929
RF	10000	0.941
RF	15000	0.947
RF	20000	0.95
RF	25000	0.953
RF	1000	0.9
RF	5000	0.932
RF	10000	0.943
RF	15000	0.947
RF	20000	0.951
RF	25000	0.954
SVM	1000	0.663
SVM	5000	0.617
SVM	10000	0.636
SVM	15000	0.672
SVM	20000	0.471
SVM	25000	0.597
SVM	1000	0.759
SVM	5000	0.26
SVM	10000	0.698
SVM	15000	0.781
SVM	20000	0.601
SVM	25000	0.778
MLP	1000	0.825
MLP	5000	0.871
MLP	10000	0.894
MLP	15000	0.906
MLP	20000	0.876
MLP	25000	0.894
MLP	1000	0.819
MLP	5000	0.862
MLP	10000	0.871
MLP	15000	0.893
MLP	20000	0.885
MLP	25000	0.905
LR	1000	0.724
LR	5000	0.745
LR	10000	0.74
LR	15000	0.739
LR	20000	0.753
LR	25000	0.755
LR	1000	0.731
LR	5000	0.743
LR	10000	0.757
LR	15000	0.726
LR	20000	0.752
LR	25000	0.748

with major advancements by Rosenblatt (1963), Marvin and Seymour (1969), and Rumelhart, Hinton, and Williams (1986).

One layer of a fully connected MLP may perform the operation $\sigma(\mathbf{w}^{\top}\mathbf{x})+b$ where $\mathbf{x}$ is an input vector, $\mathbf{w}$ is a vector of learned weights, b is a learned bias parameter, and σ is an activation function selected to improve training. The parameters of each intermediary functions are learned though a training process called back-propagation. Back propagation uses a gradient based optimizer to reduce the classification error, typically cross entropy. Training via back propagation is performed for many iterations until the model seizes to improve each iteration. MLPs are very versatile. The architecture of the MLP is the choice of number of nodes in a layer, number of hidden layers, and connectedness between layers (Goodfellow et al., 2016). These choices affect the learning capacity of the model. The greater the number of layers in a MLP, the more abstract the representation is, allowing the model more freedom to learn complicated relationships. This additional capacity also risks generating an overfit model. The Scikit-learn Python package includes a function called `MLPClassifier`. By default, the `MLPClassifier` model contains 100 hidden layers, uses the `relu` activation function, the Adam optimizer, an L2 penalty term of $\alpha = 0.0001$, a constant learning rate of 0.001, and a maximum learning of 200 training iterations (Pedregosa et al., 2011).

3.3.4. Random forest

Decision trees are the precursor to random forest classifiers. They are machine learning models represented by a tree-like flow graph. A data example is input to a trained decision tree at the root node. At the root node, some mathematical criteria determines a subsequent node the example flows through. The process of obeying a criteria to determine the path through the nodes continues until the example arrives at a leaf node. The leaf node corresponds to an output, or decision. Quinlan (1986) and Breiman, Friedman, Olshen, and Stone (1984) are regarded as two of the most significant contributions in the development of decision trees. Both algorithms seek the optimal splitting criteria in each node and yield an effective decision tree for classification (Han, Kamber, & Pei, 2012). There have been several significant advancements from the primitive decision tree. Random Forest is one of the most notable.

Ensemble methods leverage information learned in multiple models to classify with better accuracy. A typical strategy involves a hierarchical model that outputs the majority vote from each base classifier. The decision boundaries of ensembles may better reflect the *true model*. Empirically, greater diversity of base models provides better ensemble results. Random forest is an ensemble method that achieves diversity by randomizing the attributes selected in the base level tree classifiers. Random forests ensembles demonstrate among the best accuracy of the ensemble strategies for decision trees and they often faster and more robust to outliers in training data (Han, Pei, & Kamber, 2011). This study uses the scikit-learn `RandomForestClassifer` function which, by default, creates 100 estimators with the gini criterion. This study increases the number of estimators to 300 and uses the entropy metric for learning branch points.

3.4. Calculate metrics of quality

The KS inverted D statistic was calculated for each synthetic data set of 25,000. This provided a measure of similarity of the joint distribution in the synthetically generated cyber features to the baseline data set of 250,000 real examples. Random forest (RF), support vector machine (SVM), multi-layer perceptron (MLP), and logistic regression (LR) were used as the ML classifier algorithms. Each was implemented in the scikit-learn python library with default settings except for modifications noted in Section 3.3. First, each classifier algorithm was trained with the synthetic data sets (generated from 1,000–25,000 real examples). Then, each instance of the trained classifiers was tested using the baseline set of 250,000 real examples. When the classifier was trained with real examples, those examples used for training were removed from the test set. Likewise, when the classifier was trained using synthetic data, any examples used to fit the generator were removed from the test set. Classification recall was recorded, as this performance metric appropriately reports the NIDS classifier ability to avoid false negatives.

3.5. Computational experiments

Computational experiments are conducted to investigate two aspects of using synthetic data to train an ML-based NIDS. In both cases, real data is used to fit generative models. Then, synthetic data is produced from the generative models. Machine learning classifiers are trained with either real data, synthetic data, or a combination of real and synthetic data. These classifiers are tested with real labeled data.

Table 10
Classification performance of machine learning models trained with 25,000 examples of a mixture of real and CTGAN synthetic data.

% Synthetic	Classifier	Recall	Precision	F-1	% Synthetic	Classifier	Recall	Precision	F-1
0	RF	0.86128	0.862948	0.86111	50	MLP	0.73104	0.788088	0.717199
0	SVM	0.60048	0.607121	0.593908	50	LR	0.75888	0.768602	0.756614
0	MLP	0.82784	0.847363	0.825338	55	RF	0.85216	0.854406	0.851912
0	LR	0.7736	0.782306	0.771787	55	SVM	0.6768	0.676803	0.676795
5	RF	0.85952	0.861292	0.859336	55	MLP	0.8064	0.809449	0.805898
5	SVM	0.7208	0.721647	0.720505	55	LR	0.75328	0.76554	0.750324
5	MLP	0.81296	0.84265	0.808752	60	RF	0.84336	0.845898	0.843056
5	LR	0.77504	0.784431	0.773113	60	SVM	0.52832	0.531413	0.515095
10	RF	0.85936	0.861315	0.859158	60	MLP	0.80256	0.838077	0.797156
10	SVM	0.4176	0.41758	0.417532	60	LR	0.75328	0.762296	0.751077
10	MLP	0.81584	0.818133	0.815488	65	RF	0.84608	0.847542	0.845906
10	LR	0.7752	0.784699	0.773254	65	SVM	0.3552	0.355123	0.355129
15	RF	0.85888	0.860369	0.858724	65	MLP	0.81328	0.829121	0.810955
15	SVM	0.47456	0.467763	0.443362	65	LR	0.74224	0.749416	0.740309
15	MLP	0.81824	0.823286	0.8175	70	RF	0.84512	0.847428	0.844847
15	LR	0.77488	0.784299	0.772945	70	SVM	0.33408	0.331876	0.331814
20	RF	0.85792	0.859317	0.857772	70	MLP	0.75856	0.779069	0.754131
20	SVM	0.66	0.660168	0.659931	70	LR	0.73568	0.743354	0.73351
20	MLP	0.80784	0.843743	0.802612	75	RF	0.83808	0.839316	0.837921
20	LR	0.77184	0.780521	0.770007	75	SVM	0.35248	0.350074	0.349962
25	RF	0.85312	0.855023	0.85291	75	MLP	0.79744	0.824777	0.793012
25	SVM	0.72544	0.732993	0.723121	75	LR	0.7248	0.726841	0.724139
25	MLP	0.80976	0.829651	0.806789	80	RF	0.83664	0.839476	0.83628
25	LR	0.76992	0.779066	0.767963	80	SVM	0.43168	0.418719	0.407454
30	RF	0.8568	0.858879	0.85658	80	MLP	0.796	0.813252	0.793091
30	SVM	0.56704	0.579599	0.549896	80	LR	0.71584	0.717393	0.715293
30	MLP	0.80928	0.821634	0.807384	85	RF	0.82032	0.822274	0.82003
30	LR	0.77136	0.780861	0.769353	85	SVM	0.33184	0.330776	0.330735
35	RF	0.85664	0.857975	0.856496	85	MLP	0.78224	0.799548	0.77898
35	SVM	0.70832	0.720044	0.704274	85	LR	0.68208	0.683108	0.681675
35	MLP	0.8208	0.837681	0.818484	90	RF	0.8192	0.826797	0.818109
35	LR	0.76768	0.778346	0.765371	90	SVM	0.29808	0.271769	0.277097
40	RF	0.8472	0.848567	0.847039	90	MLP	0.7712	0.780189	0.769294
40	SVM	0.6464	0.649686	0.644337	90	LR	0.61408	0.62956	0.602521
40	MLP	0.79584	0.798304	0.795394	95	RF	0.79424	0.794937	0.794105
40	LR	0.76432	0.773371	0.762294	95	SVM	0.4032	0.391691	0.386545
45	RF	0.85472	0.857132	0.85446	95	MLP	0.65696	0.723911	0.629569
45	SVM	0.66128	0.663924	0.659823	95	LR	0.5208	0.584173	0.411661
45	MLP	0.79264	0.794205	0.792344	100	RF	0.49904	0.49952	0.333401
45	LR	0.76096	0.768794	0.759149	100	SVM	0.38544	0.284702	0.303624
50	RF	0.85232	0.854372	0.852093	100	MLP	0.49904	0.249041	0.332267
50	SVM	0.6464	0.646479	0.646333	100	LR	0.49904	0.249041	0.332267

3.5.1. Does quantity of real training data affect quality of synthetic data set?

This research problem leveraged the same set of 250,000 labeled real data examples collected in Maxwell et al. (2019) and maintained the same engineered features. The two methods used to generate synthetic data were CTGAN and TVAE. Each method was implemented within the Synthetic Data Vault library available for Python 3.8. Instances of the generative models were fit using random subsets of the truth data of sizes 1,000, 5,000, 10,000, 15,000, 20,000, and 25,000. These scenarios reflect a situation where the number of real training examples is the primary constraint.

One-at-a-time parameter tuning did not demonstrate any obvious improvements to the inverted KS D statistic, so default settings were adopted for CTGAN and TVAE models. The training process was repeated for a second repetition with each aforementioned model. 25,000 synthetic examples were then sampled from each trained generator model. The random subset of real samples used to train the generators of each trial were recorded in order to prevent biased ML classification at test time. That is, baseline examples used to fit the generators were removed from the classification test set.

The resulting data sets were used to train LR, MLP, RF, and SVM classifiers as described in Section 3.3. Machine learning classifiers were trained using the 25,000 examples synthetically generated from the generative models, varying the number of examples to fit the generative model as outlined above. Additionally, classifiers were trained using 1,000, 5,000, 10,000, 15,000, 20,000, and 25,000 real examples.

Finally, classifiers were trained using training sets of 1,000, 5,000, 10,000, 15,000, 20,000, and 25,000 real examples, augmented with an equivalent number of synthetic examples. The recall performance of each trained classifier is reported.

3.5.2. Does the ratio of real and synthetic data affect classification performance?

This experiment considers whether the classification performance of machine learning models is dependent on the proportion of data that is synthetic. For this experiment, it was determined that the set used to fit the generators and the classifier training sets should be much larger than in the previous experiment in Section 3.5.1. By eliminating any practical constraint on available data, stronger conclusions can be made about the research question.

A new set of real cyber data was queried from HACSAW using the method outlined in Section 3.1. 250,000 real malicious examples and 250,000 real normal examples were saved as csv files. 20% of each (malicious and normal) data set was sampled and sequestered as a test set, totaling 100,000 examples. The remaining 400,000 real examples were used to fit a CTGAN and TVAE generative model. These models were used to generate an additional 400,000 synthetic data examples. Next LR, MLP, RF, and SVM classifiers were trained with training sets ranging from 0% synthetic data to 100% synthetic data. This was done by sampling the real and synthetic data sets, without replacement, for each of the specified ratios shown in Table 2. The intent was to train models from mixed data sets as large as 400,000 examples; however, it proved computationally intractable to train a SVM classifier from very

Table 11
Classification performance of machine learning models trained with 100,000 examples of a mixture of real and CTGAN synthetic data.

% Synthetic	Classifier	Recall	Precision	F-1	% Synthetic	Classifier	Recall	Precision	F-1
0	RF	0.87622	0.877603	0.876107	50	MLP	0.82362	0.833719	0.822275
0	SVM	0.74972	0.750179	0.749605	50	LR	0.75606	0.768738	0.753149
0	MLP	0.83769	0.851893	0.836036	55	RF	0.86748	0.869031	0.867341
0	LR	0.77166	0.780512	0.769844	55	SVM	0.70201	0.715351	0.697322
5	RF	0.87592	0.877246	0.875811	55	MLP	0.79632	0.797094	0.796187
5	SVM	0.68593	0.686088	0.685863	55	LR	0.75231	0.763855	0.749571
5	MLP	0.83238	0.848432	0.830427	60	RF	0.86531	0.866984	0.865156
5	LR	0.77242	0.781517	0.770567	60	SVM	0.49758	0.497363	0.487017
10	RF	0.87511	0.876388	0.875004	60	MLP	0.82044	0.832144	0.818844
10	SVM	0.60302	0.60407	0.602016	60	LR	0.74982	0.760522	0.747224
10	MLP	0.83258	0.848945	0.830594	65	RF	0.8622	0.863822	0.862046
10	LR	0.77343	0.783161	0.771467	65	SVM	0.3535	0.347416	0.346991
15	RF	0.87474	0.875979	0.874637	65	MLP	0.80537	0.815249	0.803833
15	SVM	0.43666	0.434434	0.431839	65	LR	0.74361	0.753287	0.741138
15	MLP	0.8305	0.832302	0.83027	70	RF	0.86295	0.864619	0.862793
15	LR	0.77212	0.781755	0.770155	70	SVM	0.31978	0.315575	0.31588
20	RF	0.87384	0.875229	0.873723	70	MLP	0.81272	0.814728	0.812421
20	SVM	0.70685	0.711425	0.705256	70	LR	0.739	0.746586	0.736977
20	MLP	0.83298	0.847695	0.831194	75	RF	0.85967	0.861186	0.859523
20	LR	0.77163	0.78165	0.769581	75	SVM	0.33106	0.323257	0.323594
25	RF	0.87333	0.874788	0.873207	75	MLP	0.75282	0.781654	0.746328
25	SVM	0.49675	0.496366	0.483099	75	LR	0.72805	0.732165	0.726839
25	MLP	0.80812	0.814457	0.807148	80	RF	0.85684	0.858289	0.856695
25	LR	0.76935	0.779151	0.767307	80	SVM	0.33003	0.329746	0.32975
30	RF	0.8724	0.873676	0.872291	80	MLP	0.79727	0.80134	0.796583
30	SVM	0.50405	0.504629	0.488049	80	LR	0.71054	0.710822	0.710443
30	MLP	0.82805	0.833855	0.827299	85	RF	0.85241	0.85494	0.852147
30	LR	0.76734	0.776912	0.765312	85	SVM	0.31688	0.314417	0.314606
35	RF	0.87012	0.871466	0.870002	85	MLP	0.79315	0.793931	0.793013
35	SVM	0.44779	0.447444	0.44688	85	LR	0.67588	0.678076	0.674878
35	MLP	0.8205	0.823549	0.820076	90	RF	0.84733	0.849405	0.847103
35	LR	0.76476	0.775101	0.762528	90	SVM	0.3396	0.309026	0.312066
40	RF	0.86908	0.870513	0.868953	90	MLP	0.78571	0.787191	0.785433
40	SVM	0.71473	0.739203	0.707242	90	LR	0.60593	0.630847	0.586232
40	MLP	0.80572	0.808776	0.805238	95	RF	0.83079	0.835592	0.830183
40	LR	0.76278	0.773602	0.760411	95	SVM	0.32927	0.329267	0.329267
45	RF	0.86828	0.869703	0.868153	95	MLP	0.73751	0.750116	0.73416
45	SVM	0.70358	0.710824	0.701012	95	LR	0.53053	0.600623	0.431533
45	MLP	0.81941	0.834779	0.817313	100	RF	0.50062	0.553238	0.336735
45	LR	0.76107	0.771685	0.758713	100	SVM	0.5218	0.522069	0.520336
50	RF	0.86705	0.868682	0.866903	100	MLP	0.49962	0.301988	0.333253
50	SVM	0.6182	0.622815	0.61458	100	LR	0.5	0.25	0.333333

large sets. Therefore, classification was performed and performance was reported for training sets of 25,000 and 100,000.

4. Experimental results and discussion

4.1. Does quantity of real training data affect quality of synthetic data set?

The classification recall for each ML model was tested with real labeled examples and plotted in Fig. 3(a) for CTGAN synthetic data, Fig. 3(b) for TVAE synthetic data, and Fig. 3(c) for classification with real data features. Each plot contains 48 data points corresponding to four classifiers, six set sizes, and two repetitions. These results are provided in Tables 4–6 in Appendix. The recall of the classifiers trained on synthetic data is significantly inferior to the recall on the models trained with real examples. The effect of increasing number of training examples was investigated. The models trained with real examples demonstrate a subtle improvement in recall as the sample size increases, while the models trained on synthetic data appear to have a slight concave trend. A hypothesis test with $\alpha = 0.05$ did not conclude any significant linear trend as number of real training examples to fit the generators is increased. It is possible that the magnitude of by which number of training examples was varied was not large enough to yield a detectable difference. When real examples were used to train classifiers, the number of examples does demonstrate a significant effect on recall, with $\alpha = 0.05$.

The distributions of classification recall broken down by ML model and by data source are presented as box plots in Fig. 5. Mean classification recall for NIDS classifiers trained with real data, TVAE and CTGAN were 0.829, 0.579, and 0.517, respectively. According to Tukey's all pairs HSD test, the mean recall for each training data source (CTGAN, TVAE, real) was statistically different from each other, with $\alpha = 0.05$. Fig. 4(a) provides a graphical representation of the test. This finding somewhat contradicts the observation that CTGAN synthetic data provides a better fit to real data than TVAE synthetic data. It is therefore evident that the inverted KS D statistic is not a comprehensive metric for quality of synthetic data used to train machine learning IDS.

Mean classification recall for RF, SVM, MLP, and LR were 0.665, 0.608, 0.672, and 0.621, respectively, and Tukey's all pairs HSD test indicated insufficient evidence that treatment groups were different, with $\alpha = 0.05$. It is possible that the data sets used to fit generative models were too small to yield excellent classifiers with any of the selected algorithms. Fig. 4(b) provides a graphical representation of the test.

The best observed recall among models trained on CTGAN data, reference Table 4 and Fig. 5, was 0.699 produced by an SVM model. Interestingly, the worst observed recall, by far, was 0.308, also produced by an SVM model. The best observed recall on a model trained with TVAE, reference Table 5, was 0.725, produced by a MLP. Despite SVM producing the model with second best recall on the TVAE set, the five worst performing models were all SVM. By reasonable standards, no model trained on purely synthetic data demonstrated sufficient detection to be deployed as a NIDS.

The best observed recall among models trained on real data, reference Table 6, was 0.955, the result of a RF classifier. RF models produced excellent recall, almost always above 0.9. Though slightly

Table 12
Classification performance of machine learning models trained with 25,000 examples of a mixture of real and TVAE synthetic data.

% Synthetic	Classifier	Recall	Precision	F-1	% Synthetic	Classifier	Recall	Precision	F-1
0	RF	0.8608	0.862482	0.860627	50	MLP	0.81712	0.83306	0.814857
0	SVM	0.60048	0.607121	0.593908	50	LR	0.75488	0.758731	0.753922
0	MLP	0.82288	0.829764	0.82192	55	RF	0.85072	0.852338	0.850537
0	LR	0.7736	0.782306	0.771787	55	SVM	0.492	0.491904	0.487725
5	RF	0.85936	0.860836	0.859206	55	MLP	0.8056	0.807223	0.805325
5	SVM	0.72032	0.722429	0.719612	55	LR	0.7536	0.75739	0.752647
5	MLP	0.81696	0.840694	0.813657	60	RF	0.84576	0.847721	0.845529
5	LR	0.77344	0.782173	0.771619	60	SVM	0.46592	0.461924	0.450414
10	RF	0.85888	0.860399	0.858721	60	MLP	0.756	0.774111	0.751988
10	SVM	0.6104	0.617288	0.604347	60	LR	0.74672	0.748233	0.746304
10	MLP	0.7936	0.80106	0.792355	65	RF	0.84512	0.846105	0.845
10	LR	0.7728	0.780137	0.771253	65	SVM	0.33792	0.33554	0.335592
15	RF	0.85936	0.860501	0.859239	65	MLP	0.80496	0.805583	0.804849
15	SVM	0.6768	0.6789	0.675783	65	LR	0.73968	0.740765	0.73936
15	MLP	0.82672	0.835219	0.825581	70	RF	0.84208	0.843905	0.841857
15	LR	0.7744	0.782023	0.772815	70	SVM	0.36032	0.355999	0.355616
20	RF	0.85456	0.855711	0.854433	70	MLP	0.79424	0.806064	0.792182
20	SVM	0.42528	0.413693	0.404784	70	LR	0.74016	0.740519	0.740048
20	MLP	0.80768	0.844446	0.802331	75	RF	0.83744	0.838225	0.837336
20	LR	0.77136	0.778162	0.769905	75	SVM	0.44784	0.443121	0.436747
25	RF	0.85648	0.858453	0.85627	75	MLP	0.79072	0.790746	0.790712
25	SVM	0.56752	0.567601	0.56744	75	LR	0.72816	0.728179	0.728158
25	MLP	0.82064	0.835123	0.818636	80	RF	0.8312	0.833325	0.830914
25	LR	0.7688	0.77565	0.767305	80	SVM	0.45584	0.454276	0.450625
30	RF	0.85872	0.860376	0.858546	80	MLP	0.78432	0.787886	0.783618
30	SVM	0.48912	0.48806	0.473907	80	LR	0.72352	0.723704	0.723475
30	MLP	0.82432	0.836335	0.822697	85	RF	0.82096	0.822143	0.820782
30	LR	0.76736	0.77392	0.765909	85	SVM	0.32896	0.32528	0.325495
35	RF	0.85744	0.858654	0.85731	85	MLP	0.77872	0.778777	0.778704
35	SVM	0.6888	0.696627	0.685778	85	LR	0.71472	0.71695	0.71403
35	MLP	0.81264	0.83328	0.809636	90	RF	0.8168	0.821094	0.816159
35	LR	0.76528	0.77187	0.763798	90	SVM	0.58496	0.596478	0.571738
40	RF	0.848	0.848969	0.847885	90	MLP	0.76144	0.761799	0.76137
40	SVM	0.54944	0.55096	0.546444	90	LR	0.68288	0.693514	0.678592
40	MLP	0.7792	0.787409	0.777661	95	RF	0.7912	0.791668	0.791127
40	LR	0.76128	0.765575	0.760268	95	SVM	0.44768	0.447606	0.44755
45	RF	0.852	0.853913	0.851787	95	MLP	0.66896	0.705839	0.653682
45	SVM	0.57296	0.574488	0.570521	95	LR	0.6304	0.681694	0.602726
45	MLP	0.80912	0.818758	0.807625	100	RF	0.51584	0.721145	0.370566
45	LR	0.7568	0.76089	0.7558	100	SVM	0.6464	0.649265	0.644796
50	RF	0.85552	0.856864	0.855374	100	MLP	0.55968	0.71003	0.464502
50	SVM	0.54864	0.551933	0.540752	100	LR	0.5344	0.676049	0.418238

worse on average, MLP models also often exhibited recall better than 0.9. SVM produced the five worst performing models with recall as low as 0.677.

Fig. 6 presents the KS inverted D statistic as a function of training examples for each generative method employed and these results are tabulated in Table 7. Larger values of the inverted D statistic indicate likeness of paired variable's between two data sets (real and synthetic). The reported values are the average of the inverted D across all variables in the cyber data sets. The number of real examples used to fit TVAE and CTGAN generative models demonstrated a minor effect on data realness, as measured by the KS inverted D statistic for a comparison between synthetic and real data. Specifically, the real data benchmark included all records from the 250,000 baseline records except those used in fitting each particular generative model.

An effects test with the T distribution provides statistical evidence that increasing the number of real examples to fit TVAE also increases the KS inverted D statistic, where $\alpha = 0.05$. The results were inconclusive when performing the same effects test for the CTGAN results.

The mean inverted D statistic for data generated by CTGAN was 0.857, and it was 0.689 for data generated by TVAE. Tukey's all pairs HSD reports that there was a statistical difference in mean inverted D between generative methods. Although Chalé and Bastian (2021) previously reported an inverted D statistic for CTGAN, TVAE and Markov Chain Monte Carlo generative methods of 0.890, 0.723, and 0.875, respectively, these reflect a different cyber dataset containing only one discrete variable and two continuous variables while only training the generators with 10,000 examples. Therefore, this suggests it is significantly more difficult to achieve the same goodness of fit in generating synthetic cyber data when working with higher-dimensional data sets. The HSD Tukey test is shown graphically in Fig. 4(c).

The evaluation of recall was extended to classification models trained by a mixture of half real and half generated data. This strategy depicted scenarios where 1,000, 5,000, 10,000, 15,000, 20,000, and 25,000 real examples were available. By incorporating an equal quantity of synthetic data, the models were trained with 2,000, 10,000, 20,000, 30,000, 40,000, and 50,000 total training examples. An effects test was performed using the T distribution and $\alpha = 0.05$. The test concluded that there is no statistical evidence that the number of real examples to fit the generator was important for the classification recall of machine learning models. This was true for the half real, half synthetic mixed datasets derived from both CTGAN and TVAE.

Fig. 7(a) presents the recall for models trained with equal quantities of real examples and synthetic data generated from CTGAN and the results are tabulated in Table 8 in the Appendix. The nine best performing models were all RF, each achieving a recall of at least 0.932, even when the number of real training examples were as few as 5,000. The nine worst performing models were SVM.

Fig. 7(b) presents the recall for models trained with equal quantities of real examples and CTGAN synthetic examples. These results are tabulated in Table 9. The top 10 performing models were all RF, each achieving a recall of at least 0.929. No RF model had a recall below 0.896. Only one ML and one LR model demonstrated recall

Table 13
Classification performance of machine learning models trained with 100,000 examples of a mixture of real and TVAE synthetic data.

% Synthetic	Classifier	Recall	Precision	F-1	% Synthetic	Classifier	Recall	Precision	F-1
0	RF	0.87591	0.877315	0.875794	50	MLP	0.8254	0.827653	0.825099
0	SVM	0.74972	0.750179	0.749605	50	LR	0.757	0.76242	0.755739
0	MLP	0.83173	0.832603	0.83162	55	RF	0.86753	0.868997	0.867398
0	LR	0.77166	0.780512	0.769844	55	SVM	0.65011	0.652111	0.648955
5	RF	0.87624	0.877588	0.876129	55	MLP	0.82245	0.834392	0.82085
5	SVM	0.6767	0.679434	0.675464	55	LR	0.75234	0.755808	0.751498
5	MLP	0.83796	0.838393	0.837908	60	RF	0.86583	0.867307	0.865695
5	LR	0.77168	0.78013	0.769945	60	SVM	0.3571	0.356919	0.356897
10	RF	0.87569	0.876921	0.875588	60	MLP	0.81962	0.819842	0.819589
10	SVM	0.72021	0.720278	0.720188	60	LR	0.74907	0.751466	0.748471
10	MLP	0.83608	0.843322	0.835211	65	RF	0.86276	0.864202	0.862624
10	LR	0.77087	0.778666	0.769256	65	SVM	0.63858	0.641806	0.636512
15	RF	0.8748	0.875917	0.874707	65	MLP	0.81363	0.816683	0.81318
15	SVM	0.67152	0.672277	0.671159	65	LR	0.74422	0.745579	0.743866
15	MLP	0.83806	0.842775	0.837501	70	RF	0.86186	0.86339	0.861714
15	LR	0.77122	0.778762	0.769662	70	SVM	0.60751	0.612717	0.602924
20	RF	0.87385	0.875101	0.873745	70	MLP	0.81486	0.829806	0.812738
20	SVM	0.684	0.684435	0.683814	70	LR	0.73863	0.739157	0.738486
20	MLP	0.83916	0.842129	0.83881	75	RF	0.85972	0.86103	0.859593
20	LR	0.77151	0.779099	0.769946	75	SVM	0.6834	0.685757	0.682393
25	RF	0.87315	0.874479	0.873037	75	MLP	0.81643	0.823988	0.815353
25	SVM	0.6459	0.646108	0.645774	75	LR	0.73059	0.730598	0.730588
25	MLP	0.82613	0.82613	0.82613	80	RF	0.85515	0.856177	0.855045
25	LR	0.77047	0.777684	0.76897	80	SVM	0.36897	0.368522	0.368432
30	RF	0.87252	0.873743	0.872416	80	MLP	0.80488	0.824369	0.801904
30	SVM	0.6509	0.65115	0.650756	80	LR	0.72086	0.721919	0.720527
30	MLP	0.82881	0.850089	0.826169	85	RF	0.85319	0.854954	0.853007
30	LR	0.76886	0.776213	0.767311	85	SVM	0.36626	0.365845	0.365769
35	RF	0.87002	0.871328	0.869905	85	MLP	0.77594	0.780054	0.775114
35	SVM	0.39139	0.390622	0.39032	85	LR	0.70517	0.709791	0.703537
35	MLP	0.83008	0.832795	0.829733	90	RF	0.84246	0.843811	0.842305
35	LR	0.76681	0.773516	0.765372	90	SVM	0.49334	0.492599	0.480334
40	RF	0.86943	0.870759	0.869313	90	MLP	0.76951	0.775411	0.768269
40	SVM	0.63677	0.641583	0.633656	90	LR	0.68507	0.70202	0.678323
40	MLP	0.82729	0.836042	0.826158	95	RF	0.82947	0.831371	0.829225
40	LR	0.76324	0.769293	0.761902	95	SVM	0.48383	0.481648	0.468017
45	RF	0.86826	0.869553	0.868145	95	MLP	0.71482	0.72963	0.710146
45	SVM	0.69244	0.702131	0.688709	95	LR	0.63285	0.691331	0.602474
45	MLP	0.82674	0.827924	0.826584	100	RF	0.51887	0.717038	0.376561
45	LR	0.7605	0.765972	0.759262	100	SVM	0.4821	0.480433	0.470826
50	RF	0.86697	0.868471	0.866834	100	MLP	0.52281	0.663836	0.391964
50	SVM	0.64127	0.641397	0.64119	100	LR	0.5241	0.67278	0.393658

surpassing 0.9. SVM was by far the worst performing classifier for data sets combining equal proportions of real and TVAE generated data.

Tukey's all pairs HSD test with $\alpha = 0.05$ was used to test statistical difference between the mean recall of classifiers trained with real data, CTGAN data, TVAE data, real data augmented with CTGAN data, and real data augmented with TVAE data. The test indicated no evidence that the classifiers trained with mixed data sets performed differently than the models trained with real data. The classifiers trained with only synthetic data had significantly worse recall. Table 3 presents the connection table of mean recall for all groups under test and Fig. 8 provides a graphical representation of the test.

4.2. Does the ratio of real and synthetic data affect classification performance?

Classification recall is presented in Fig. 9 for classifier models where the training data is a mixture of real and CTGAN generated data and the percent synthetic is varied. Similarly, the recall for models trained with a mixture of real and TVAE generated data is presented in Fig. 10. The experiment was performed once with training sets containing 25,000 examples, Figs. 9(a) and 10(a), and again with training sets containing 100,000 examples, Figs. 9(b) and 10(b). Each plot contains 84 data points corresponding to four classifiers, crossed with 21 settings of percent synthetic. The full results for these trials are presented in Tables 10–13 in the Appendix.

An effects test with $\alpha = 0.05$ determines that *percent synthetic* is an important factor for estimating recall. Further analysis indicates that *percent synthetic* may also have a quadratic effect, however the quadratic term is not conclusively significant on all data sets, with $\alpha = 0.05$, and the model adequacy checks suggest that residuals may not be independent. The results indicate that varying the training set size between 25,000 and 100,000 does not have a significant effect on recall. The mean recall of models trained on 25,000 examples (Tables 10 and 12) is 0.715 and the mean recall of models trained on 100,000 examples (Tables 11 and 13) is 0.730. When these two treatments are considered categorical, Tukey's HSD test fails to conclude the mean recall is different, with $\alpha = 0.05$. The low influence of training set size may be because the ML classifiers have a finite learning capacity.

Overall, classification recall is greatest with training sets containing no synthetic data and the recall decreases slightly as the percent synthetic increases. Figs. 9–10 show an inflection point around 85% synthetic, at which point the recall decreases more rapidly. Following the standard practice of right sizing data sets in principal component analysis, this inflection point indicates that 15% real examples should be retained in training sets for machine learning based intrusion detection systems. An effects test with $\alpha = 0.05$ also determines that the machine learning classifier is an important factor for estimating recall. For every set in this experiment (25,000 CTGAN, Table 10; 100,000 CTGAN, Table 11; 25,000 TVAE, Table 12; 100,000 TVAE, Table 13), RF performed the best, and SVM performed the worst. The best observed recall in this experiment was 0.876 and the 20 best performing models were all RF.

The recall observed in this experiment for RF trained with 25,000 real examples is 0.861 (observed in both Tables 10 and 12). This is

inferior to the recall of 0.955 observed in Section 3.5.1 (see Table 6) where the RF model was also trained on 25,000 real examples. The lower recall in this experiment may be due to the increased size of the test set or random events during RF training.

The performance of RF had only decremented slightly upon offsetting 85% of the data with synthetic data. The recall for the RF models trained on 85% synthetic data were 0.820, 0.852, 0.821, and 0.853 with the 25,000 CTGAN, 100,000 CTGAN, 25,000 TVAE and 100,000 TVAE training sets, respectively.

Across the four sets, the mean recall for RF, SVM, MLP, and LR is 0.839, 0.535, 0.789, and 0.729, respectively. The standard deviation of recall for RF, SVM, MLP, and LR is 0.069, 0.069, 0.076, and 0.138, respectively. Evidently, the spread of recall for trials using the SVM classifier is much greater than for other classifiers.

Figs. 9 and 10 show that despite an overall trend of decreasing recall with increasing synthetic data, the relationship between recall and percent synthetic data is more nuanced for the SVM classifier. The SVM classifier used in this study employs the radial basis function kernel, which struggles to converge with large, diverse training sets. SVM may also be the least compatible classifier with the feature engineering scheme, shown in Table 1. This may be because SVM cannot perceive a difference between numerically similar, but distinct, label encodings. It is unclear why SVM classification improves slightly around 50% synthetic and again at 100% synthetic data.

The results in this work show that some machine learning classifiers perform well when trained on sets of partially synthetic training data. This finding may help alleviate the lack of quality labeled cyber data for NIDS research. Deep learning classifiers have higher capacity than the conventional ML methods used in this study. Deep models excel with large, complex feature sets but they also demand a massive quantity of training data. The discovery that synthetic data can be used to offset real data lends itself to the possibility that deep methods can be applied with even greater classification performance.

5. Conclusion

It is difficult to overstate the urgency of leveraging AI and ML technologies for cybersecurity to help protect military networks from attack. If vulnerabilities are not monitored, adversaries can suppress friendly systems from operating or potentially execute kinetic effects. NIDS, in conjunction with continuous monitoring, is an excellent way to detect, analyze, and react to threats.

As demonstrated in this work, the CTGAN and TVAE generative methods can each generate synthetic cyber data reasonably well. However, ML models trained with purely synthetic data resulted in underwhelming classification recall. These models yielded an unacceptable rate of false negatives. Classifiers trained with half synthetic data and half real data performed statistically equivalently to the classifiers trained with only real data. Further, it is shown that there is a linear relationship between percent synthetic data and classifier performance. NIDS classifiers performed well when trained with at least 25,000 examples and 15% being real. The engineered features used in this study did not use minimax scaling or one hot encoding for all variables, and this choice may have led to decremented classification performance with some algorithms.

Generative methods, including machine learning, evolutionary computation and Bayesian approaches, are an area of ongoing growth. The fast rate of advancement in this field is motivated by the broad applications of synthetic data. Synthetic cyber data, in particular, promises to address three well-documented issues with NIDS data in advanced cybersecurity research: lack of relevant records, expandability as new information arises, and statistically rigorous testing and evaluation. Despite the efficacy of using synthetic data to train ML-based NIDS, as demonstrated in this research, there are some drawbacks. Chalé and Bastian (2021) critique that the joint distribution of synthetic cyber data produced by CTGAN and TVAE lacks likeness to the real data from the multivariate perspective. Therefore, non-conventional metrics such as Energy Statistics, proposed by Székely and Rizzo (2013), may provide insight unto the likeness of complex synthetic data sets to the baseline distribution. There is also a theoretical limitation on the utility of high quality synthetic data. Synthetic data from generative models does not introduce additional predictive information the system, but does improve statistical learning with the information already contained in real data. In the absence of a complex physical model or simulation, this limitation will likely not be breached.

We look forward to exploring improvements to generative methods as it may be applied to cyber data generation and evaluation, particularly for adversarial sample generation. These advancements will enable research on vulnerabilities and failure modes in AI-based NIDS. Future research should conduct cyber subject-matter-expert analysis of ML classifier false positives to determine if the classifiers can actually discover attacks that were mislabeled in the test set. These studies may expand beyond real cyber data onto high fidelity simulated data from cyber test beds such as LARIAT (Braje, 2016) and CyberVAN (Chadha, Bowen, Chiang, Gottlieb, Poylisher, Sapello, Serban, Sugrim, Walther, Marvel, Newcomb, & Santos, 2016). Future work will build upon the previous work of Alhajjar et al. (2021), Bierbrauer, Chang, Kritzer, and Bastian (2021), Devine and Bastian (2021), Schneider, Aspinall, and Bastian (2021), Talty, Stockdale, and Bastian (2021) to investigate adversarial sample generation in the constrained domain, as well as the use of meta-learning for improving the robustness of AI-based NIDS. Further extensions will explore evolutionary machine learning by developing cyber data generative methods through the open-world recognition context to generate not-yet-seen, but realistic and functional cyber data that is dynamic to potential future adversarial behavior.

CRediT authorship contribution statement

Marc Chalé: Data curation, Methodology, Investigation, Formal analysis, Writing – original draft, Visualization, Funding acquisition. **Nathaniel D. Bastian:** Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Security Agency Laboratory for Advanced Cybersecurity Research under Interagency Agreement No. USMA21035, the U.S. Army Combat Capabilities Development Command (DEVCOM) Army Research Laboratory under Support Agreement No. USMA21050, and a GEN Omar N. Bradley Foundation Officer Research Fellowship in Mathematics. The authors would like to express great gratitude to Casey Lorenzen, Jedd Gibney and the team at the U.S. Army Engineer Research and Development Center who supported our research by allowing use of their Cybersecurity Environment for Detection, Analysis and Reporting (CEDAR). Special thanks is also given to Dr. Bruce Cox for providing conceptual background in generative machine learning and for discussing experimental results. The views expressed in this paper are those of the authors and do not reflect the official policy or position of the Air Force Institute of Technology, United States Military Academy, the United States Air Force, the United States Army, the Department of Defense, or the United States Government.

Appendix

See Tables 4–13.

References

Alhajjar, E., Maxwell, P., & Bastian, N. (2021). Adversarial machine learning in network intrusion detection systems. *Expert Systems with Applications*, *186*, Article 115782. http://dx.doi.org/10.1016/j.eswa.2021.115782, URL: https://www.sciencedirect.com/science/article/pii/S0957417421011507.

Anderson, J. P. (1972). Computer security technology planning study—Vol 1. HQ Eletctronic Systems Division (AFSC), Deputy for Command and Management Systems, Bedford, Massachusetts.

Anderson, J. (1980). Computer security threat monitoring and surveillance.

Axelsson, S. (2000). Intrusion detection systems: A survey and taxonomy. *Technical report*.

Azmoodeh, A., Dehghantanha, A., & Choo, K.-K. R. (2019). Robust malware detection for internet of (battlefield) things devices using deep eigenspace learning. *IEEE Transactions on Sustainable Computing*, *4*(1), 88–95. http://dx.doi.org/10.1109/TSUSC.2018.2809665.

Bejtlich, R. (2013). *The practice of network security monitoring: understanding incident detection and response*. No Starch Press.

Bierbrauer, D., Chang, A., Kritzer, W., & Bastian, N. (2021). Cybersecurity anomaly detection in adversarial environments. In *Proceedings of the AAAI fall 2021 symposium on AI in government and public sector* (pp. 1017–1022). URL: https://arxiv.org/abs/2105.06742.

Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *COLT '92, Proceedings of the fifth annual workshop on computational learning theory* (pp. 144–152). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/130385.130401.

Braje, T. M. (2016). Advanced tools for cyber ranges. MIT Lincoln Laboratory Lexington United States.

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Classification and regression trees. *Group*, *37*(15), 237–251.

Brent, L. (2019). Nato's role in cyberspace. URL: https://www.nato.int/docu/review/articles/2019/02/12/natos-role-in-cyberspace/index.html.

Chadha, R., Bowen, T., Chiang, C.-Y. J., Gottlieb, Y. M., Poylisher, A., Sapello, A., et al. (2016). Cybervan: A cyber security virtual assured network testbed. In *MILCOM 2016 - 2016 IEEE military communications conference* (pp. 1125–1130). http://dx.doi.org/10.1109/MILCOM.2016.7795481.

Chae, H. S., Jo, B. o., Choi, S. H., & Park, T. k. (2013). Feature selection for intrusion detection using nsl-kdd. *Recent Advances in Computer Science*, *20132*, 184–187.

Chalé, M., & Bastian, N. (2021). Challenges and opportunities for generative methods in cyber domain. In *2021 winter simulation conference*. IEEE.

Chalé, M., Bastian, N. D., & Weir, J. (2020). Algorithm selection framework for cyber attack detection. In *WiseML '20, Proceedings of the 2nd ACM workshop on wireless security and machine learning* (pp. 37–42). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/3395352.3402623.

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, *41*(3), http://dx.doi.org/10.1145/1541880.1541882.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 321–357.

Chen, T. M., & Robert, J.-M. (2004). The evolution of viruses and worms. *Statistical Methods in Computer Security*, *1*, 1–16.

Chollet, F. (2021). *Deep learning with python*. Simon and Schuster.

Cicalese, C. (2013). Redefining information operations. National Defense University Fort McNair.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, *20*(3), 273–297.

Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, *20*(2), 215–232.

Denning, D. (1987). An intrusion-detection model. *IEEE Transactions on Software Engineering*, *SE-13*(2), 222–232. http://dx.doi.org/10.1109/TSE.1987.232894.

Denning, D., & Neumann, P. G. (1985). *Requirements and model for IDES-a real-time intrusion-detection expert system, vol. 8*. SRI International Menlo Park.

Devine, S. M., & Bastian, N. D. (2021). An adversarial training based machine learning approach to malware classification under adversarial conditions. In *Proceedings of the 54th Hawaii international conference on system sciences* (pp. 827–836). ScholarSpace.

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). BayesIan data analysis. CRC Press.

Goodfellow, I. (2017). NIPS 2016 Tutorial: Generative adversarial networks. arXiv:1701.00160.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press, http://www.deeplearningbook.org.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., et al. (2014). Generative adversarial nets. In *Advances in neural information processing systems* (pp. 2672–2680).

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., et al. (2020). Generative adversarial networks. *Communications ACM*, *63*(11), 139–144. http://dx.doi.org/10.1145/3422622.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining concepts and techniques* (3rd ed). Waltham, MA: Morgan Kaufmann.

Han, J., Pei, J., & Kamber, M. (2011). *Data mining: concepts and techniques*. Elsevier.

Hastings, W. K. (1970). Monte Carlo sampling methods using Markov chains and their applications. Oxford University Press.

Hindy, H., Brosset, D., Bayne, E., Seeam, A. K., Tachtatzis, C., Atkinson, R., et al. (2020). A taxonomy of network threats and the effect of current datasets on intrusion detection systems. *IEEE Access*, *8*, 104650–104675. http://dx.doi.org/10.1109/ACCESS.2020.3000179.

Hinton, G. E., Sejnowski, T. J., & Ackley, D. H. (1984). *Boltzmann machines: constraint satisfaction networks that learn*. Carnegie-Mellon University, Department of Computer Science Pittsburgh, PA.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An introduction to statistical learning, vol. 112. Springer.

Japkowicz, N. (2000). The class imbalance problem: Significance and strategies. In *Proc. of the int'l conf. on artificial intelligence, vol. 56*. Citeseer.

Japkowicz, N., Myers, C., Gluck, M., et al. (1995). A novelty detection approach to classification. In *IJCAI, vol. 1* (pp. 518–523). Citeseer.

Jebara, T. (2012). *Machine learning: discriminative and generative, vol. 755*. Springer Science & Business Media.

Kass, R. E., Carlin, B. P., Gelman, A., & Neal, R. M. (1998). Markov chain Monte Carlo in practice: a roundtable discussion. *The American Statistician*, *52*(2), 93–100.

Kerwin, K. R., & Bastian, N. D. (2020). Stacked generalizations in imbalanced fraud data sets using resampling methods. *The Journal of Defense Modeling and Simulation*, *18*(3), 175–192. http://dx.doi.org/10.1177/1548512920962219.

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114.

Kingma, D., & Welling, M. (2014). Auto-encoding variational Bayes. In *Proceedings of the international conference on learning representations*.

Lashkari, A. H., Kadir, A. F. A., Taheri, L., & Ghorbani, A. A. (2018). Toward developing a systematic approach to generate benchmark android malware datasets and classification. In *2018 international carnahan conference on security technology* (pp. 1–7). IEEE.

Law, A. M., & Kelton, W. D. (2007). *McGraw-Hill series in industrial engineering and management science, Simulation modeling and analysis* (4th ed.). McGraw-Hill Book Co., New York.

Lim, S. K., Loo, Y., Tran, N.-T., Cheung, N.-M., Roig, G., & Elovici, Y. (2018). Doping: Generative data augmentation for unsupervised anomaly detection with gan. In *2018 IEEE international conference on data mining* (pp. 1122–1127). IEEE.

Marvin, M., & Seymour, A. P. (1969). Perceptrons. MIT Press.

Massey Jr, F. J. (1951). The Kolmogorov-Smirnov test for goodness of fit. *Journal of the American Statistical Association*, *46*(253), 68–78.

Maxwell, P., Alhajjar, E., & Bastian, N. D. (2019). Intelligent feature engineering for cybersecurity. In *2019 IEEE international conference on big data (Big Data)* (pp. 5005–5011). IEEE.

McHugh, J. (2000). Testing intrusion detection systems: a critique of the 1998 and 1999 darpa intrusion detection system evaluations as performed by lincoln laboratory. *ACM Transactions on Information and System Security*, *3*(4), 262–294.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, *21*(6), 1087–1092.

Miller, K. C., O'Halloran, B., Pollman, A. G., & Feeley, M. K. (2019). Securing the internet of battlefield things while maintaining value to the warfighter. *Journal of Information Warfare*, *18*(2), 74–84, II–III, URL: https://afit.idm.oclc.org/login?url=https://www-proquest-com.afit.idm.oclc.org/scholarly-journals/securing-internet-battlefield-things-while/docview/2424656410/se-2?accountid=26185, Copyright - Copyright Peregrine Technical Solutions 2019; Last updated - 2021-04-02.

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to linear regression analysis, vol. 821*. John Wiley & Sons.

Patki, N., Wedge, R., & Veeramachaneni, K. (2016). The synthetic data vault. In *2016 IEEE international conference on data science and advanced analytics* (pp. 399–410). IEEE.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, *1*(1), 81–106. http://dx.doi.org/10.1007/BF00116251.

Rosenblatt, F. (1963). A bibliography of perceptron literature. *Scientific and Technical Information*, 189.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536.

Russell, S., & Abdelzaher, T. (2018). The internet of battlefield things: the next generation of command, control, communications and intelligence (C3I) decision-making. In *MILCOM 2018-2018 IEEE military communications conference* (pp. 737–742). IEEE.

Salvatier, J., Wiecki, T. V., & Fonnesbeck, C. (2016). Probabilistic programming in Python using PyMC3. *PeerJ Computer Science*, *2*, Article e55.

Scharre, P. (2018). *Army of none: autonomous weapons and the future of war*. WW Norton & Company.

Schneider, M., Aspinall, D., & Bastian, N. (2021). Evaluating model robustness to adversarial samples in network intrusion detection. In *Proceedings of the 2021 IEEE international conference on big data*. in press.

Serinelli, B. M., Collen, A., & Nijdam, N. A. (2020). Training guidance with KDD cup 1999 and NSL-KDD data sets of ANIDINR: Anomaly-based network intrusion detection system. *Procedia Computer Science*, *175*, 560–565.

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, *6*(1), 1–48.

Siwek, J. (2021). Zeek::HTTP. URL: https://docs.zeek.org/en/master/logs/index.html.

Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. Colorado Univ at Boulder Dept of Computer Science.

Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE symposium on security and privacy* (pp. 305–316). http://dx.doi.org/10.1109/SP.2010.25.

Stallings, W., Brown, L., Bauer, M. D., & Bhattacharjee, A. K. (2012). Computer security: principles and practice. Pearson Education Upper Saddle River, NJ, USA.

Stolfo, S. J., Wei Fan, Wenke Lee, Prodromidis, A., & Chan, P. K. (2000). Cost-based modeling for fraud and intrusion detection: results from the JAM project. In *Proceedings DARPA information survivability conference and exposition DISCEX'00, vol. 2* (pp. 130–144). http://dx.doi.org/10.1109/DISCEX.2000.821515.

Székely, G. J., & Rizzo, M. L. (2013). Energy statistics: A class of statistics based on distances. *Journal of Statistical Planning and Inference*, *143*(8), 1249–1272.

Talty, K., Stockdale, J., & Bastian, N. (2021). A sensitivity analysis of poisoning and evasion attacks in network intrusion detection system machine learning models. In *Proceedings of the 2021 IEEE military communications conference* (pp. 1017–1022). IEEE.

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD cup 99 data set. In *2009 IEEE symposium on computational intelligence for security and defense applications* (pp. 1–6). http://dx.doi.org/10.1109/CISDA.2009.5356528.

United States Department of the Army (2010). Cyberspace operations concept capability plan.

Ventre, D. (2020). *Artificial intelligence, cybersecurity and cyber defense*. John Wiley & Sons.

Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. *Advances in Neural Information Processing Systems*, *32*, 7335–7345.

Yildirim, I. (2012). BayesIan inference: Metropolis-hastings sampling. Dept. of Brain and Cognitive Sciences, Univ. of Rochester, Rochester, NY.